B.Comp. Dissertation

Enhancing customizability of the voting component of the Orbital portal

By
Tan Wei Zhe

Department of Computer Science
School of Computing
National University of Singapore
2024

B.Comp. Dissertation

Enhancing customizability of the voting component of the Orbital portal

By
Tan Wei Zhe

Department of Computer Science
School of Computing
National University of Singapore
2024

Project No: H204100
Advisor: Senior Lecturer Zhao Jin

Abstract

A customizable voting system has been implemented in Skylab v2 to not only support the voting event at Splashdown but also other potential types of voting events. The system offers customization features such as candidate displays, role weights, and results customization, which can be tailored for different use cases. To support customization, the system is designed with high-level ideas like separating customization into the different voting stages and low-level ideas like isolating customizable components and config-based factories. The system underwent several phases of user study to not only identify issues and improve through a contextual inquiry but also to evaluate the primary customization features through a survey guided by the Technology Acceptance Model (TAM). Various forms of good software engineering practices were also applied to ensure the quality of the system.

Subject Descriptors:

- Software and its engineering---Software creation and management---Designing software---Software implementation planning
- Software and its engineering---Software creation and management---Designing software---Software design engineering
- Human-centered computing---Human computer interaction (HCI)---HCI design and evaluation methods---User studies

Keywords:

Software engineering, software usability, user studies, customizability

Implementation Software:

React, Express, Postgres, Next.js, Node.js, Prisma

Acknowledgement

I would like to express my gratitude to my advisor, senior lecturer Zhao Jin, for all of his guidance and feedback throughout the project. Their feedback has significantly shaped my work and understanding of the subject.

Table of contents

Chapter 1: Introduction

Orbital is a yearly program of the National University of Singapore (NUS), that allows students to propose, design, execute, implement, and market software projects. It provides peer evaluation, critique, and presentation milestones over the summer period. At the end of Orbital, **Splashdown**, which is the closing event for Orbital, will be held physically, and it consists of an online voting process, a project presentation session, invited talks, and an awards ceremony. Students would participate in Orbital in teams of two, where they would work on their proposed project.

The Orbital portal, **Skylab**, is used to support the process. The development of Skylab was completed in 2016 and the website has not changed much since then. However, the scale and requirements of the Orbital program have continued to grow since then. To better support this, the development of Skylab v2 started in 2022. Most of the core features of the website have been completed but it has yet to be released for actual usage as it is still in development. There are 4 roles in Skylab v2. Student, which is the basic role a student participating in orbital has. Advisor, a more experienced student who is in charge of a few project teams evaluates and provides feedback on the projects. Mentors are often external guests who represent key companies in the industry and local start-ups but can also come from the school in the form of senior students, including Orbital alumni. Lastly, administrators, who have the highest access level in Skylab v2. A user can have multiple roles, for example, an advisor can also be a student.

One thing that both Skylab and Skylab v2 lacked was the support for an in-built voting system; multiple external software are used to execute the current voting process. To shift away from the reliance on external software and to have a comprehensive portal that can support Orbital throughout the whole process, Skylab v2 needs to have its voting system. If a voting system were to be designed for Skylab v2, it should not only be able to handle the current Splashdown voting process but also a variety of other use cases. To achieve this, the voting system has to be customizable offering different options for different cases. This is also required for future iterations of the voting process in Orbital.

The main aim of this project is to **implement a customizable voting system in Skylab v2**. The customizable voting system should be able to be customized to best fit different voting processes with primary and secondary customization options that are justified through existing literature and the study of similar voting systems.

1.1 Project Objective Description

1. Identify primary and secondary customizable features [Completed]
Primary customizable features are important customizations that are mainly identified through the review of existing literature and systems, including the current voting process at Splashdown. Secondary customizable features are complementary features that are also identified through similar means but are more straightforward as compared to primary customization. The **primary customizable features** are split into three categories: **the way candidates are displayed to voters, the weights of the different roles of voters,** and **the choice of what to display in the results.**

2. Come up with a design that supports customizability [Completed]
To have a good design that supports customizability, I have followed many software engineering principles, with the **main guiding principle being the Open/Closed Principle (OCP)**. OCP ensures that the system can scale with any modifications made to the customization, which allows for constant evolution of the system. The design also draws inspiration from existing systems and how they handle customizability.

3. Iteratively improve the usability of the system and evaluate primary customizable features through user studies [Completed]
Multiple rounds of user study were conducted to **identify usability issues and improve the system**. In each user study, a **contextual inquiry** was conducted with the participants, where they had to perform a fixed set of tasks with the system. The feedback and observations were then **systematically analyzed using affinity diagramming and prioritized** before the improvements were made to the system. Subsequent user studies helped to evaluate the changes and identify new areas for refinement, with a combination of new and returning participants for a good mix of opinions. At the end of each user study, participants completed a **survey** that evaluated the primary customization features of the system through the **Technology Acceptance Model (TAM)**.

Chapter 2: Background

Designing a customizable voting system requires identifying which **customization options** are useful. To achieve this, it's essential to review both **existing literature and systems** to understand the tried and tested features and their reasonings. Inspirations will also be taken from how the **design of existing systems supports customizability**. Additionally, understanding the **current voting process** used by Orbital, which has already been proven to work, provides a foundation for implementing improvements that align with user needs.

2.1: Literature Review

The different aspects of voting covered in this literature review are voter registration and weighted voting. The aim of this literature review is to explore more into some aspects of voting to see the potential for customization.

Voter registration

There are many ways to conduct voter registration, and the importance of customizability in this area is uncovered through existing literature. Burden and Neiheisel (2013) studied the assumption that voter registration negatively impacts turnout and found that registration itself does not have much impact on turnout. The other factors of registration like closing date and administrative capacity played a bigger role in affecting turnout (Burden et al, 2013). Supporting this, Street, Murray, Blitzer, and Patel (2015), revealed through the analysis of web search data that extending registration deadlines up to the voting day itself can boost the number of registered voters and also voter turnout. They show that much of the interest in registering was concentrated in the last few days of the campaign duration. We can see that customizability in the administrative process of registering voters and the flexibility of the registration dates are crucial.

Automatic registration of voters is one of the ways to conduct voter registration, besides having voters manually register, and it has been shown to have positive outcomes. Barnes and Rangel (2014) present a case study on Chile, where there was a major law reform in 2012, causing the country to switch to automatic registration and voluntary voting. Chile not only grew its electorate but also saw an increase in turnout among the youth (Barnes et al, 2012). Additionally, in the case of the United States, automatic voter registration has yielded higher

rates of registration and, more specifically, increased the rate of registration among minorities in states that have adopted this style of registration (McGhee 2019). This shows that automatic registration is a good alternative to manual registration in certain cases, and customization would be good in allowing administrators to select their preferred choice of registration.

Overall, these studies are important in showing the impacts of voter registration on voting, and how those impacts can be adjusted by modifying the different aspects of voter registration. By offering options such as manual registration with flexible timings, or allowing administrators to add voters directly to the valid voter list, customizable voter registration can cater to varying levels of voter engagement and administrative capacity.

Weighted voting

The weighted voting section discusses the potential of weighted voting. Weighted voting involves customization of the different weights a voter has, and this in turn will affect the results of the vote. Kurz, Maaser, and Napel (2016) analyzed the proportionality between constituency size and voting weight and concluded that approximate proportionality to the square root of population sizes or direct proportionality may be optimal, depending on voter's preferences. These weights were to be assigned to representatives of their constituencies to provide a fairer representation. What we can learn from this is that a voting system should allow the customization of weights based on some role a voter has to potentially give them a more appropriate representation of the importance of the role.

Similarly, Yusifov (2018) explored e-voting systems and analyzed the benefits and disadvantages of weighted voting in democratic systems, where voters who have more experience and are more capable will have higher voting weights in the approach of weighted voting. They state that political parties will offer more realistic policies and relevant candidates, ultimately leading to a more representative and effective government, with the help of weighted voting. This further supports the use of weighted voting. However, they acknowledge that assigning voting weights according to the status level of a person can be seen as unfair.

The examination of weighted voting highlights the importance of customization in voting systems, where voters can be assigned custom weights based on their importance. This leads

to a fairer representation of the votes in the results and ultimately a more robust voting system.

In conclusion, this literature review explains what customization is important in the various aspects of voting, including voter registration and weighted voting. This customization allows for a more adaptive voting system that can handle multiple scenarios. It can be customized to meet different requirements and provide other benefits as explained in the literature above.

2.2: Existing Systems

In addition to existing literature, there are systems out there that support voting and have already been used by many organizations to hold their voting events. This section will analyze three very popular voting websites, ElectionBuddy, ElectionRunner, and StrawPoll. By analyzing these systems, we can gain insights into valuable customization features and usability principles for our system.

ElectionBuddy (https://electionbuddy.com/)

+ Multiple ways for administrators to add voters (manual entry, import CSV, copy from another election, import from external application)
+ Can randomize candidate order
+ Can select what results statistics to display
- Candidate display is in list view only

ElectionRunner (https://electionrunner.com/)

+ Multiple ways for administrators to add voters (manual entry, import CSV)
+ Multiple voting methods available (multiple choice, ranked choice)
+ Can randomize candidate order
- Published results are final and cannot be modified

StrawPoll (https://strawpoll.com/)

+ Offers image polls
+ Live results
+ Can select when to publish results
- The results display is fixed

Analysis of Key Customization Features

First of all, all three display systems in our customizable voting platform allow you to add text and images to voting options, **enabling control over what information is shown** to voters and how it appears. By default, options are displayed in a table, a common format in existing systems. In our case, voters will be selecting projects in Orbital, and our system can make use of project data already stored in the database. Each project typically includes a poster that provides an overview, which our system can use as the voting image. Unlike existing systems that require you to manually input all information for each voting option, our platform can draw directly from the database. The potential customization here is to let administrators choose what to display to voters.

Secondly, other than StrawPoll, the existing systems allow you to add weights to voters, which can **change the worth of the votes** voters cast. The importance of weighted voting was also highlighted in the literature review section. However, existing systems require manually setting weights for each voter, which is tedious and error-prone, especially when there is a large number of voters. On the other hand, the roles in Orbital offer a good way to categorize the voting weights as each role contributes differently. Our system can simplify the process by allowing customization of weights by roles instead of individual voters.

Lastly, this is a feature that only ElectionBuddy has, which is the ability to show or hide certain statistics in the results. The other two existing systems have more or less fixed result displays. This is another potential area of customization for our system. Customizing the results can **change the results so that they can be more aligned with the voting objectives**.

There are also many other features like importing voters through a CSV file, randomizing candidate order, and setting the number of votes per voter that can be incorporated into our system, but in terms of customizability, the more important ones are mentioned above.

Design Insights from Existing Systems

Adapting good customization features from existing systems is not sufficient, we must also understand how customizability is supported in the design of the systems. In the existing systems, there is a **centralized area** where administrators can configure the vote event settings and view the event data. ElectionRunner and ElectionBuddy logically separate them into different tabs, making navigation more intuitive. For example, there is a tab for voters, a

tab for ballots, and a tab for results. ElectionBuddy, in particular, arranges tabs in the logical sequence of the voting process, streamlining setup.

Figure 1: Administrator view of tabs in ElectionBuddy

By **grouping up related configurations**, administrators can easily customize the different aspects without being distracted by the other options. Our system will follow such a structure, balancing structure and flexibility, which enhances customizability while having a user-friendly interface.

2.3: Current Voting Process of Orbital

To build a good voting system, it is also highly important to understand what is going on in the current voting process that Orbital uses. Voters are authenticated by a unique voting ID (number + check digits) which is first generated using a spreadsheet. For those that are registered (students and staff), a script will be used to email voter IDs together with the details of Splashdown, while the public will be issued their IDs on the spot at Splashdown. The voting is done through Google Forms, where voters key in project IDs, and the data will be extracted for consolidation. Next, voting data has to be validated (check for valid voter ID, duplicate votes, etc), before formulas are used to apply weights and the final result is sorted. This is also done using a spreadsheet.

The shift to Skylab v2 means that most of the voting process can be handled by its voting system. This is not limited to but includes, the management of voters, the casting of votes, the validation of votes, and the consolidation of results. We can reduce the use of external software in such cases and also manage the voting process in a more centralized way.

One improvement that can be made is that, since the voting system is on Skylab v2, we can make use of the fact that users already have an account there to authenticate them and use the information available in the system to easily add them as voters or give them voting weights based on roles. They will be referred to as internal voters. Considering Splashdown where guests of the event can also vote but will not have an account, we will classify them as

external voters and they are identified through voter IDs. Not only would it be an inconvenience to them, but having them register an account on a platform for a one-time event would be a waste of resources. Internal voters will have to log in to vote while external voters authenticate with the voter ID given to them manually.

2.4: Summary

In this chapter, multiple perspectives were explored to lay the foundation for the design of a customizable voting system for Orbital. The literature review shows the importance of voter registration and weighted voting. Flexible deadlines and automatic registration can benefit voter turnout, while weighted voting can more accurately represent the relative importance of different voter roles. This highlights the high potential of making these two aspects customizable in our system.

Next, existing systems were also analyzed, with the display of voting options, role weights and results customization being the core features that we would like to have as customization features in our system. I went beyond just identifying these features, to also looking at gaps the existing systems have in these areas and how our system can be better. For example, utilizing existing data that our system has in the database. The design of how our system supports customizability also takes inspiration from some of the existing systems. Having customization be divided into their respective logical groupings would be beneficial.

Finally, the current voting process was reviewed. There is a reliance on many external tools and much of the process like the consolidation of results has to be done manually by the administrator. Our system can help to centralize the process and also offer a more efficient way of managing data with what it has in the database. For example, the process of managing internal voters is more streamlined as the database already has the data of the system's users.

This chapter sets the direction for how our customizable voting system will be built, with effective customization and design supported by existing literature, systems, and processes.

Chapter 3: Approach

This chapter will explain the approach to coming up with a customizable voting system. First, the system overview will be given, which includes an explanation of the architecture, the 5 main voting stages, and the primary customization features. It will then go into important designs in the frontend and backend that support the customization.

3.1: System Overview

This section will first go through the general architecture of the system, together with the tech stack. It will then introduce the 5 main stages of the voting process, which is also the high-level design of the system, followed by the 3 primary customization features.

3.1.1: General Architecture

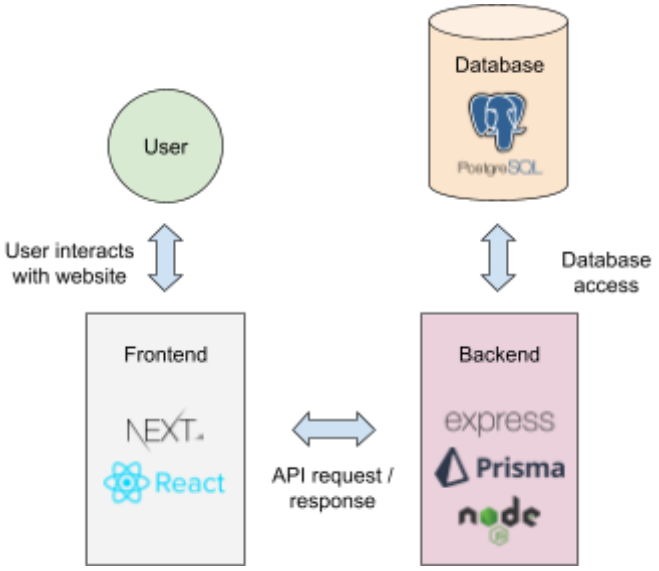


Figure 2: General architecture including core frameworks used

Skylab v2 uses the PERN (Postgre, Express, React, Node) stack. The frontend uses Next.js and React, which contain a tree of functional components. The components will make API calls to the backend which uses Express and runs on Node.js. In the process of handling the API requests, the backend might read or write to the database which is a PostgreSQL database. Prisma is the framework that acts as an abstraction layer over the database, developers interact with Prisma for database access.

3.1.2: Voting Process Stages

The system is designed to provide administrators with end-to-end control over vote events. Following existing systems, the system would have a **centralized area** for administrators to manage the vote event, this will be known as the manage vote event page. Existing systems also handle the huge amount of customizations present by separating them into logical groups. To do this, I separated the voting process, from the view of the administrator, into **5 stages**.

Stage 1: Creation of vote event

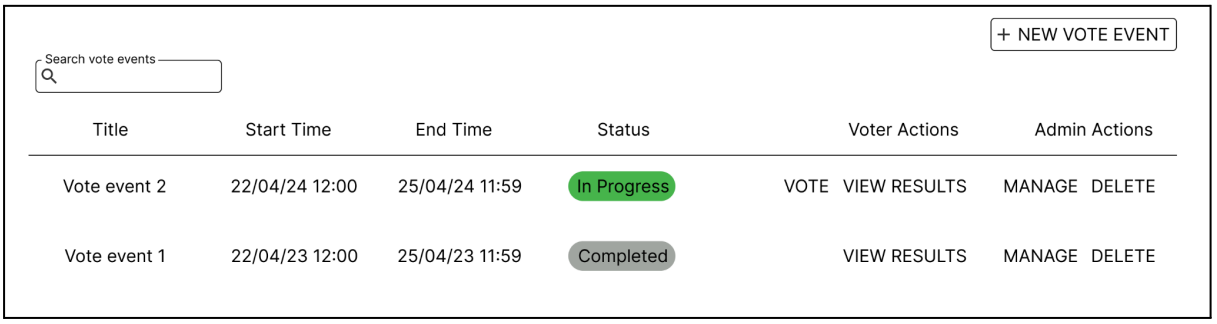


Figure 3: Administrator view of the vote events page

To even configure the vote event, administrators have to first create a vote event. The next four stages are logically grouped into their respective tabs on the manage vote event page.

Stage 2: Selection of voters

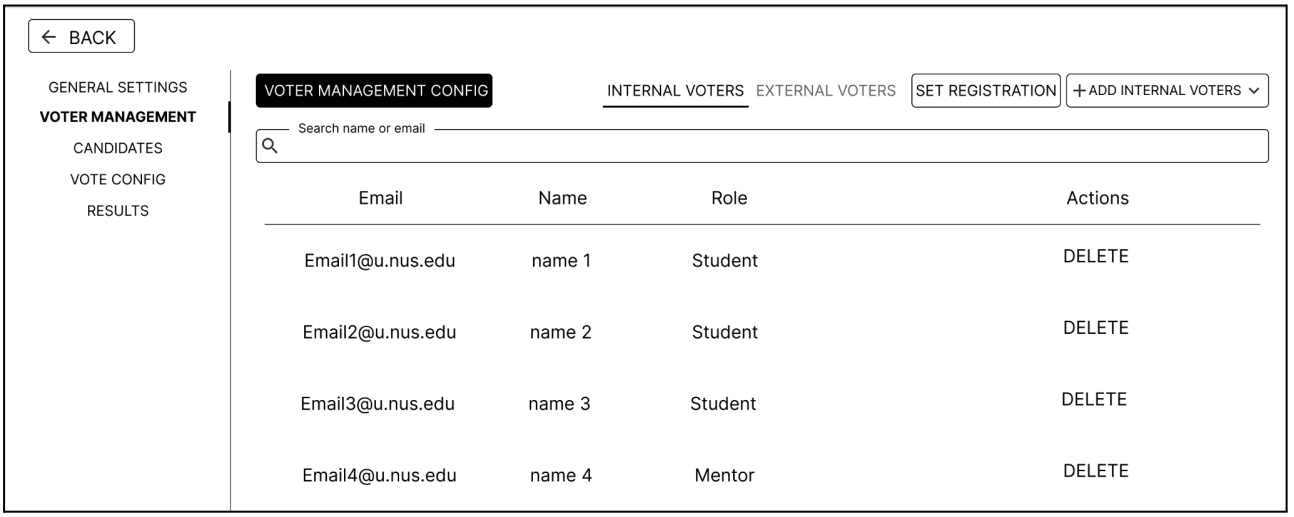


Figure 4: Voter management tab of the manage vote event page

Administrators can manage voters in this tab, with many different ways to add voters available. Adding voters to the vote event can be seen as selecting voters. One of the secondary customization features which is the ability to select the type of voters is found in the voter management config. The tabs follow a **standardized layout**, with config-type buttons on the top left and action buttons on the top right. Each tab will have a table containing information relevant to the tab, with a search bar above to filter. This provides a sense of **familiarization between tabs** and reduces big UI shifts or confusion when switching tabs. Furthermore, the tabs are also ordered according to the **natural order** of the stages in the voting process, which allows for a smooth flow of navigation.

Stage 3: Selection of candidates

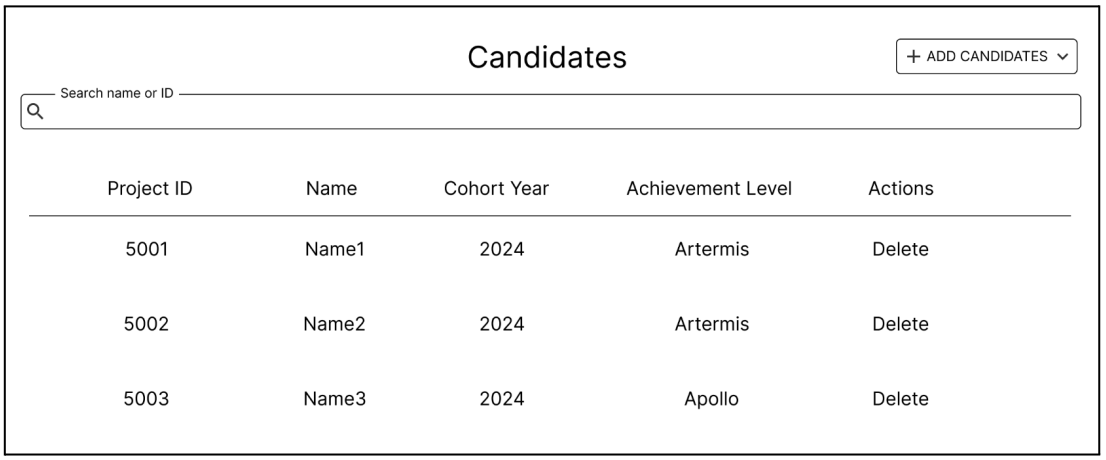


Figure 5: Candidates tab of the manage vote event page

Administrators can add candidates to their vote event from this tab. Candidates is the general voting term used to refer to Orbital projects. No customization is available in this tab as of now, but it is possible to extend this in the future.

Stage 4: Configuration of Voting Style and Display

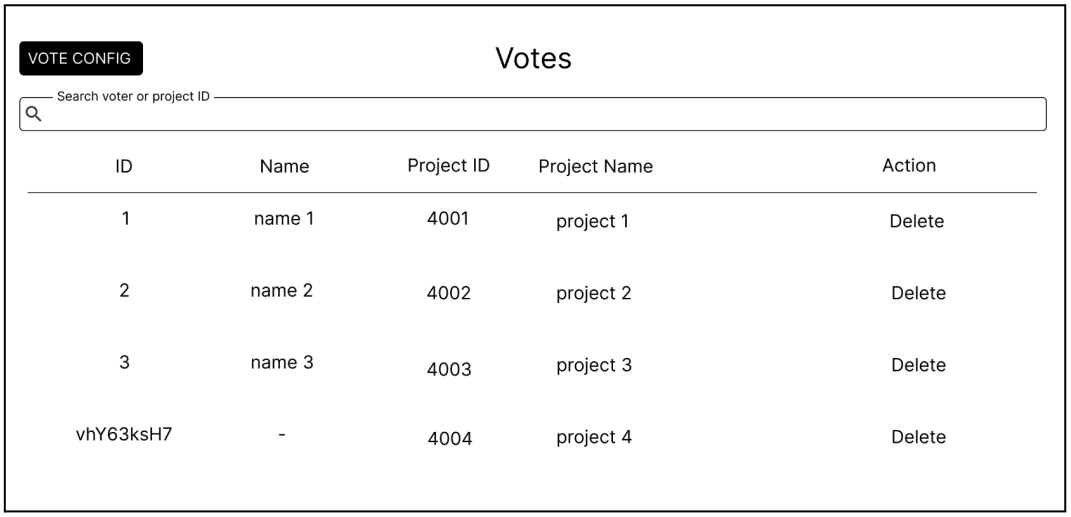


Figure 6: Vote config tab of the manage vote event page

Administrators can configure the voting style and display it in the vote config tab. The vote config button opens a modal where the following can be set: (1) candidate display type, (2) minimum and maximum votes per voter, (3) randomization of candidate order, and (4) voting instructions. All of these are secondary customization features except for candidate display.

Stage 5: Consolidation and release of results

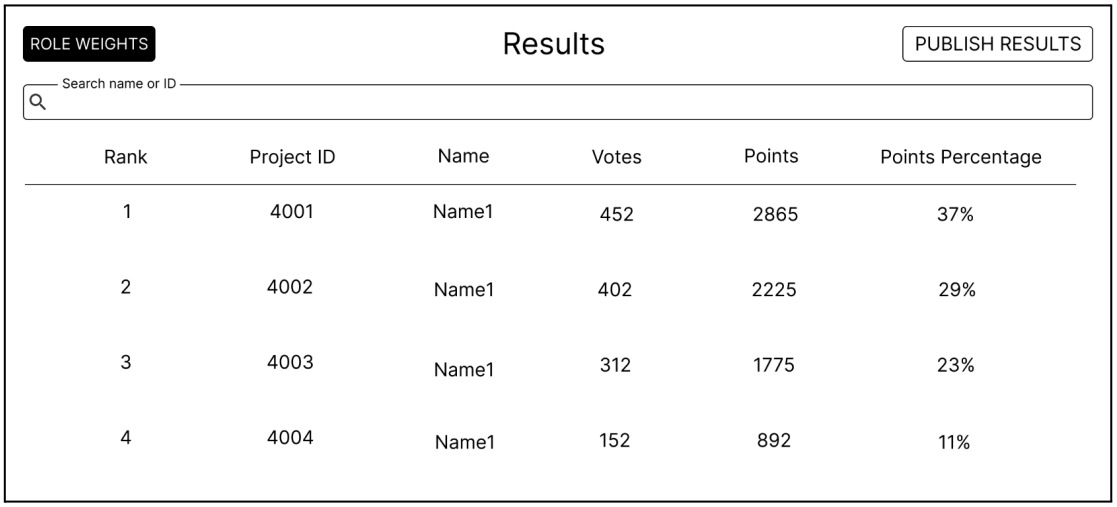


Figure 7: Results tab of manage vote event page

Administrators can set the roles weights, and also publish a customized version of the results for voters to view, in this tab.

3.1.3: Primary Customization Features

This section will go through the three primary customization features, which can be found in two stages of the voting process: stage 4 and stage 5. Since the voting options in our system are candidates, the display of voting options will be referred to as the candidate display. Actual system screenshots can be found in the appendix (D - System Screenshots of Primary Customization Features).

Candidate display (stage 4)

Candidate display is the way candidates are displayed to voters. The current voting process at Splashdown uses Google Forms and **does not display any projects** in the form. This is possible because Orbital Projects have posters that give an excellent summary of the project, and these posters are available for voters to see physically at Splashdown. As such, voters already have the required information and can easily just fill in the project IDs they want to vote for. On the other hand, I proposed having a voting page that displays the posters in the form of a gallery so that voters can view the project information and vote at the same time. This is to handle the case where there isn't any information on the candidates given to the voters beforehand or the case where the voter wants to recall some information on certain projects. This will be referred to as the **gallery view**. The final candidate display is the **table view display** that is used by default in the existing systems, it's a view that has been proven to work in the systems. It balances the information available by not giving too much information on the project but enough to let voters know what are the available projects. In the case of the table view and gallery view, voters can vote by directly selecting projects.

Role Weights (stage 5)

Weights can be given to the different roles that voters have in the system. Internal voters can have the following roles: administrator, mentor, advisor, and student. If an internal voter has multiple roles, they will be assigned one role in order of how the roles were introduced earlier, based on the roles that they have. External voters are assigned the public role only. The weights of these 5 roles can be customized by the administrator. They can use this to decide the importance of each role in the vote event because every voter is given the same number of votes.

Results customization (stage 5)

Administrators can customize the results of vote events they publish by choosing to show or hide the following: rank, number of votes, number of points, and point percentage. Project ID and name will always be shown. They can also choose to limit the results to the top few candidates. Customizing this depends on the intended goal of the vote event. By limiting the results to the top few, you can focus the attention on them without having to reveal those who did not score well. By not showing the number of votes and points at the same time, you can hide the fact that roll weights were used as points are votes multiplied by the weights. By hiding the rank, you can avoid showing that certain candidates perform better. For example, if I want to have a vote event to choose 10 representatives, then I can hide everything and limit the results to 10, and the voters will just see the 10 representatives in the results without anything to differentiate them. Ultimately, administrators can set the transparency level of the vote event based on its objectives.

3.2: Frontend Design

The frontend is designed to accommodate the system's customizable requirements. To achieve this, I followed a component-driven design and utilized various design patterns, such as the **factory pattern,** to handle configurable user interfaces (UI). The main guiding principle behind the designs is the **Open/Closed principle (OCP)**, which ensures that modules are open for extension but closed for modification. This is important for customizable systems because customizable features can be extended without modifying existing code. OCP allows customization options to evolve easily while keeping the rest of the system stable. Complementing this is the technique of **progressive disclosure** that is also applied to manage complexity and improve user experience.

3.2.1: Component Driven Design

React apps are made up of components, where each piece of the UI has its logic and appearance. A component can be as small as a button, or as large as an entire page. (*Quick Start – React*, n.d.). The frontend follows a component-driven design approach, which breaks down the UI into small, independent, and reusable pieces. The components are then put together using component composition, which is one of the core concepts of React that helps us to create sophisticated UIs by putting together small and independent components (Bhimani, 2023). This is especially important in a customizable system with many moving

parts, where different components fit together based on the customization applied. It adheres to OCP by allowing higher-level components to be extended without modification, and Separation of Concerns, where the many different functionalities are split into their components.

Voting page example

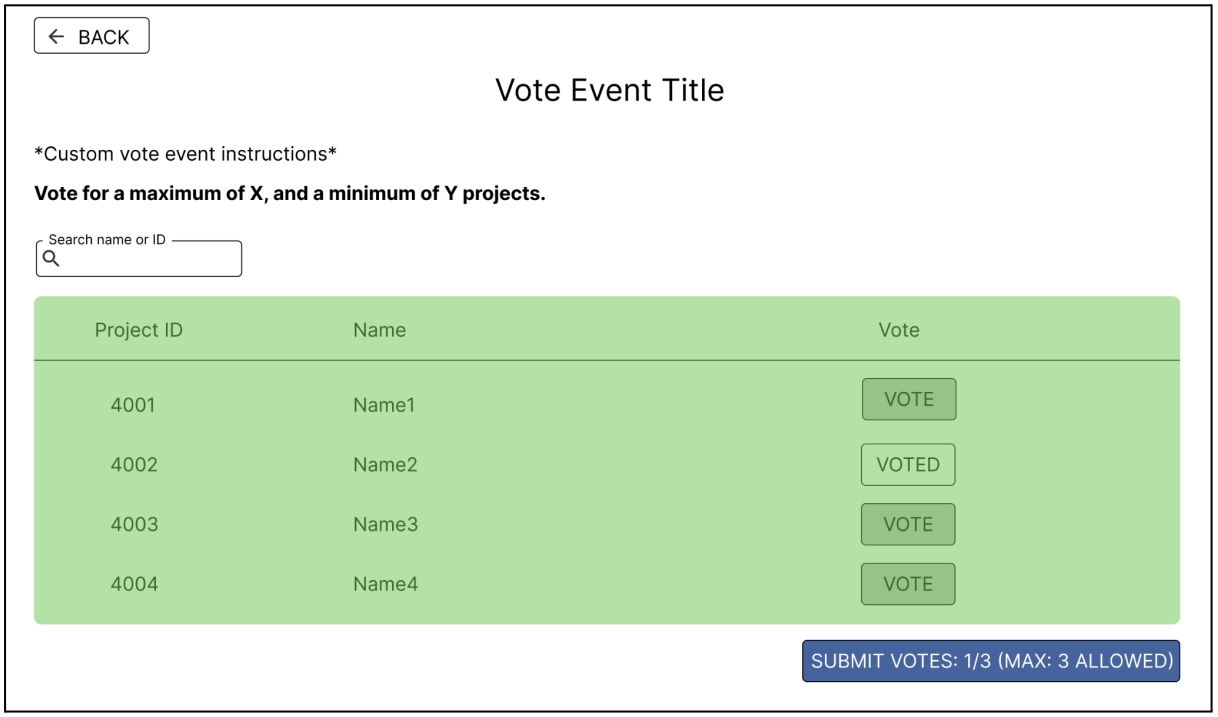


Figure 8: Voting page

In the voting page, the green area represents the candidate display component which is dynamically generated by a factory. It is designed in such a way that the candidate display component is only responsible for displaying what it needs to display and updating the selected candidates. The submit votes modal, which is opened by clicking the button, has the responsibility of sending the votes to the backend, and the typography and search components have their self-explanatory responsibilities. This allows customization of the candidate displays while keeping the rest of the components constant on the voting page. In contrast, designing a whole voting page for each candidate display type would result in more work and repeated code for common functionalities.

The main idea here that can be applied to customizable parts of the system is to **isolate the changing parts** from the rest of the components, and ensure that it does not have any extra

responsibility. It allows customization to scale smoothly, independent of common functionalities.

3.2.2: Factory Pattern

Although OCP is already present with proper component separation, I would like to take it up to another level and introduce it to how components are composed. When it comes to a customizable system, customization options should be flexible, options can be easily added, modified, or removed based on the requirements. As such, the main parts of the system that have the highest potential to undergo extension are the customizable ones. OCP is used here to ensure that the components that compose the customizable aspects of the system do not need to undergo modification when the customization is extended.

To achieve this, the factory pattern is used. The factory pattern is a creational design pattern that encapsulates the logic of creation, decoupling it from the code that uses the created item (Shah, 2024). There will be component factories that are responsible for returning the components that are related to the customizable options, supporting customizable component rendering based on user-selected options.

Version 1 design:
The initial factory design created factories for each component type, responsible for generating components based on a configuration object. Each factory contained hard-coded logic to render components with associated props, which led to repeated code across factories. This design allowed customizable UI options but lacked flexibility, as each factory had to be individually modified for changes or additions, violating the DRY principle and resulting in tightly coupled code.

Version 2 design
There is now a function createDynamicComponentFactory(config: FactoryConfig) that takes in a factory config and returns a factory defined by the given factory config. A factory config consists of a list of component config which defines the components the factory will generate.

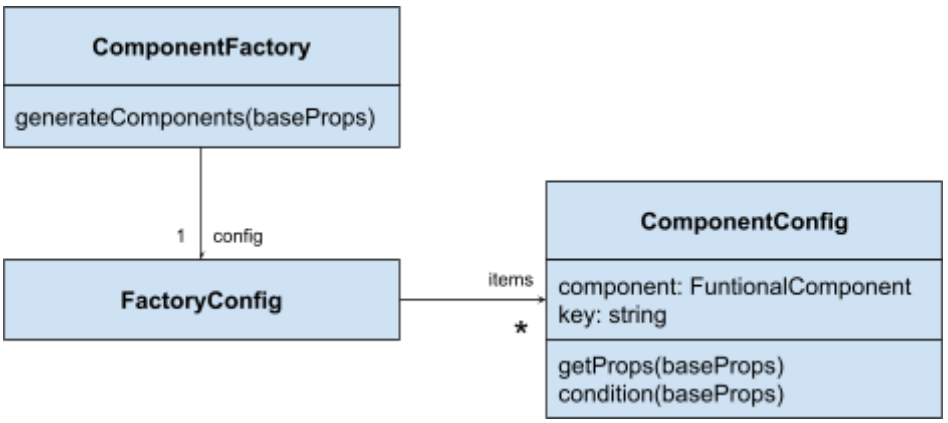


Figure 9: Class diagram representation of version 2 factory design

ComponentFactory: The factory itself, which generates components based on the given FactoryConfig.

FactoryConfig: The configuration object that contains an array of ComponentConfig entries.

ComponentConfig: Defines each item's component, key, props generator, and optional condition.

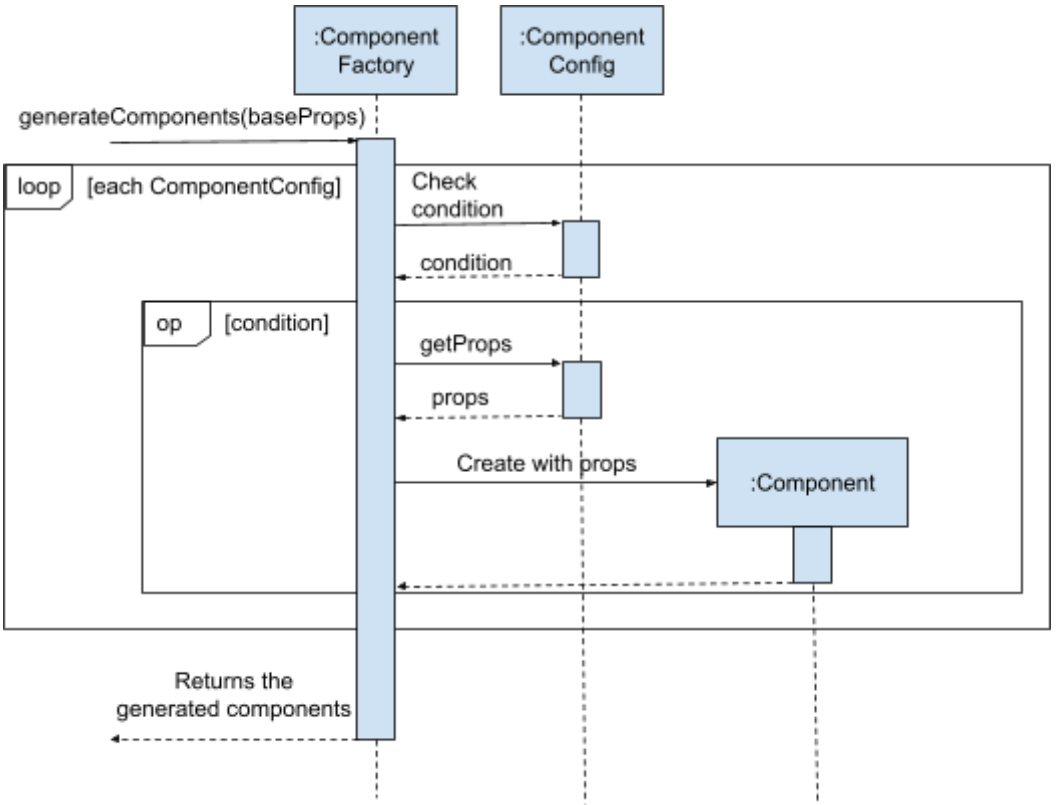


Figure 10: sequence diagram of how factories generate components

The generateComponents function in each component factory iterates over the ComponentConfig entries in the FactoryConfig. For each entry, it checks whether the condition for rendering is met. If true, it fetches the required props and creates the component, returning all generated components.

Advantages of version 2 design over version 1:

1. **No longer a need to define individual factories with potentially repeated code**. Only need to define a factory config and call the createDynamicComponentFactory function with it to get the required factory. This can be done for all new factories in the future.
2. Components can be added or removed by **simply modifying the configuration object** without changing the factory logic itself. For example, adding a new menu item or display type is as simple as adding a new configuration entry. This is highly important for customizable systems.
3. The condition function allows us to easily **control when a component should be rendered based on runtime conditions**, making it highly adaptable to changing requirements.

Comparison to the common Factory pattern in OOP

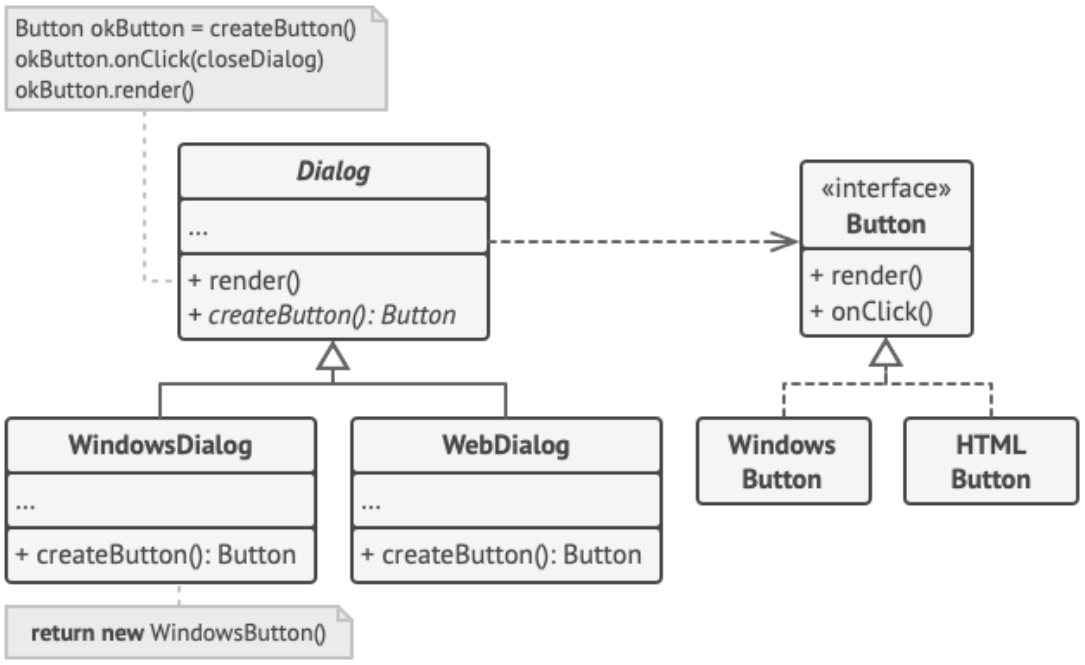


Figure 11: An example of the Factory pattern. From "Factory Method", Refactoring.Guru, 2024, (https://refactoring.guru/design-patterns/factory-method)

This is an example of the basic factory pattern. The factory in this case is the abstract Dialog class which is inherited by the WindowsDialog and WebDialog classes. The Dialog class can be seen as a factory that creates buttons and different buttons can be created by simply creating the appropriate class that inherits from Dialog and their respective createButton method. Version 2 of my factory pattern offers many more advantages compared to this basic factory pattern.

1. It allows for **dynamic customization of components through configuration objects** (FactoryConfig and ComponentConfig). This means that adding new components or changing their behavior is as simple as updating the configuration, without altering or subclassing any code. In the traditional OOP factory pattern, you will need to write new subclasses or override factory methods to achieve similar flexibility.
2. The **declarative style** of returning components makes it easier for developers to understand what the UI will look like based on certain configurations, rather than following imperative logic or flow of instantiations like in OOP.
3. (getProps: (baseProps) => any) function allows the factory to assign dynamic props to components based on runtime data (baseProps), allowing for even more **customization at the component level**.
4. With the ability to apply conditions (condition: (baseProps) => boolean), my factory can **dynamically decide whether to generate** certain components based on runtime conditions.

Usage of the factory pattern

There are currently 4 factories present in the codebase. Factories are used to generate the different add voters and candidates option components. The factory currently generates all of the options, but the options can be made customizable in the future. The voting page also uses a factory to generate the relevant candidate display component. The component is generated based on the candidate display set in the vote config, which is handled by the condition function in the component configs.

By using the factory pattern, customizable parts are easily modified and extended without altering existing code. It allows more useful options to be easily added in the future or a change in the current ones, which leads to scalable customization in the system.

3.2.3: Centralized Mapping of Customization Options

There is a DISPLAY_OPTIONS object that maps a candidate display type enum value (None, Table, Gallery), to its component and its description. This object not only serves as a clear grouping of candidate display options, but also ensures type safety and provides a **centralized way to manage display logic**. Any future changes to the display options can be easily made by adding or modifying entries in this object.

This was done due to how the folder structure of components is organized, which is based on the UI element type of the component. For example, the table candidate display component would be in the tables folder while the gallery candidate display option would be in the grids folder. As the code base or the customization options expand, it would get increasingly hard to track the components related to customization.

This can be extended to future customization options as a way to improve their maintainability.

3.2.4: Progressive Disclosure

When it comes to customizability, there can be times when there are many different options to choose from, and it can result in increased complexity which negatively affects user experience. There are two important laws used in UX design. **Hick's Law** states that the **time it takes to make a decision increases with the number of options**. **Miller's Law** states that on average, **a person's working memory can only hold about 7 (plus or minus 2) items at a time**.

To help reduce the complexity, progressive disclosure is used. Instead of presenting all the information upfront, the additional details will only be revealed when necessary. In 3.1.2 Voting Process Stages, you can see the first layer of progressive disclosure in the way the tabs are divided, something that I learned from existing systems. Information and settings relevant to each stage of the voting process are hidden away in their respective tabs. You only need to access what you need without being overwhelmed by the rest. The second layer of progressive disclosure is present inside each tab, where configs and actions are hidden in their respective modals and menus. They are only accessed when needed.

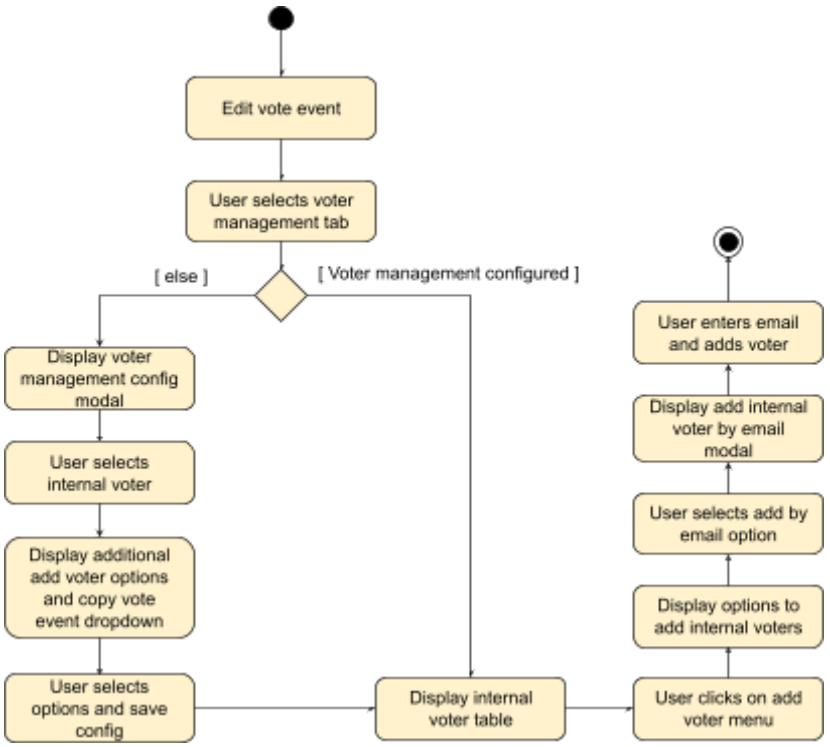


Figure 12: Activity diagram of how administrators can add an internal voter

From this activity diagram, information is only shown to the user when required. Without proceeding with the required step, the user will not be burdened with the unneeded information. In this case, the user does not need to bother with the information from candidates, vote config and results, or other actions like importing voters through CSV.

3.3: Backend Design

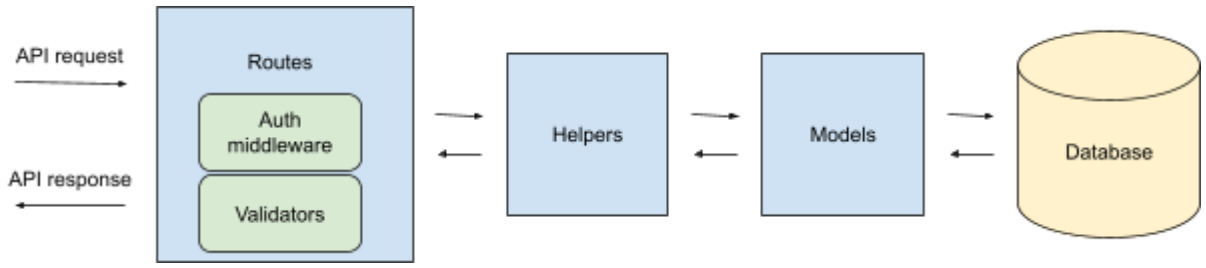


Figure 13: Flow of data in the backend

The backend has a design similar to a layered architecture. When a request arrives it will go through the route, helper, and model layers. A layered architecture also follows OCP by allowing the extension to target a specific layer without modifying the rest. This is a design that is set in place by the previous developers, so I will not be going into detail.

3.3.1: Database Design

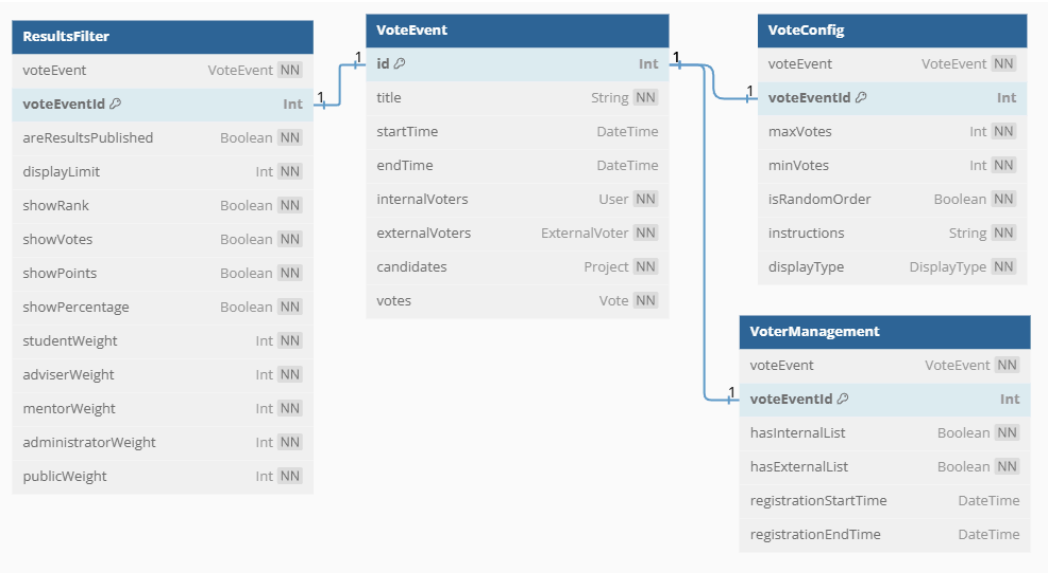


Figure 14: Entity-relationship diagram for vote events

The database is structured to **logically separate configuration data** based on the stages of a voting event. This organization enhances both the maintainability and efficiency of the database, as it allows each API call to target only the specific stage required for a request. The stage-based separation aligns the backend and the frontend and simplifies the data management process as compared to one giant relational table for vote events. Customization changes can be **localized to their respective stages**, without affecting the other stages.

Chapter 4: User Study

The two aims of the user study are to **(1) identify usability issues and areas for improvement, and (2) measure how effective the primary customization features are**. The first goal was achieved through **ethnographic research, using contextual inquiry**. The second goal was achieved through a **survey**.

4.1: Methodology

The user study was conducted **iteratively to have a constant feedback loop** on the different versions of the system. A total of 3 phases was conducted on versions 2, 3, and 4 of the system respectively. This aligns well with the iterative model approach used for the development.

Participants tested the voting system individually on the Skylab v2 website. The testing was done on a local version of the site run from my laptop and took about thirty minutes to one hour per participant. Participants first took part in the contextual inquiry before finishing the survey at the end. Each participant was given an identifier to maintain anonymity.

Target audience
School of computing students or have participated in Orbital in one of the following roles (student, adviser, mentor, or administrator). A minimum age of 19 years is required for NUS students and 21 years for the rest. Participants should be physically healthy.

4.1.1: Contextual Inquiry

Contextual inquiry is a type of customer interview where participants perform tasks to show how they use a product or service (Amaresan, 2020). Participants completed a set of tasks while I observed their actions. A voice recording was done with Zoom for the duration of the contextual inquiry not only to ease the data collection process but to also serve as solid evidence. As such, participants were asked to articulate their actions and the reasons behind them. There were a total of 6 tasks (appendix C - User Study Tasks) that participants had to complete, and I would have a quick discussion after each task to gather feedback and ask questions based on my observations. Contextual inquiry allowed for **real-time observation**

of user interactions, revealing usability issues and providing insights for improvement based on authentic use patterns.

Before starting, apart from the participant information sheet and consent form, participants were provided with a document that contained all the context and definitions relevant to the tasks they would perform. It ensures the participants have sufficient knowledge to play the roles relevant to each of the tasks, mimicking a user of the system as closely as possible. This helped me collect more reliable data.

Seeding of database

The contextual inquiry requires the database to be pre-populated with certain data, and all participants should begin the contextual inquiry with the same database state. To **automate** this process, a seed was created to easily refresh the database to have the required data. Prisma comes with an integrated seeding functionality, and all I had to do was provide it with the relevant seed files. After writing the code for the seed files, I was able to easily refresh the database for each participant with a single command. This seeding functionality was also utilized in testing.

4.1.2: Survey

The aim of the survey (link) conducted after the contextual inquiry is to find out how effective the customization features of the voting system are. The survey will make use of one of the most popular models of technology acceptance, the **Technology Acceptance Model (TAM)** (Davis, 1989). The basis of TAM is that the users' behavioral intention determines the acceptance and use of a specific technology. The two main components that drive behavioral intention are perceived usefulness (PU) and perceived ease of use (PEOU). The tool used for the survey was Google Forms.

The 3 primary customization features, (1) the different candidate displays for the voting page, (2) the ability to apply weights to the different roles, and (3) the choice of what to display when publishing results, are what will be the main focus of the survey. I measured the users' acceptance of these features using the two variables of PU and PEOU.

The survey will mainly be conducted using a quantitative approach, using a questionnaire to determine the PU and PEOU of each primary customization. Most of the questions will be using a 5-point Likert-like scale.

Survey Versions and Rationale

The survey went through two rounds of refinement, with version 3 of the survey being used in the user study.

Version 1: Originally, each customization option had one PU and one PEOU question. This version was too simplistic, offering limited insight.
Version 2: To provide more depth, this version grouped each customization feature into a single unit, with three PU and three PEOU questions each.
Current Version (Version 3): For the candidate display feature, each option (no display, table view, gallery view) is evaluated individually with three PU and three PEOU questions each. This approach allows me to capture perceptions of each display, while other features remain assessed as in Version 2.

Survey questions

The questions are adapted based on the 12 questions proposed by Davis

1. Using [this product] in my job would enable me to accomplish tasks more quickly.
2. Using [this product] would improve my job performance.
3. Using [this product] in my job would increase my productivity.
4. Using [this product] would enhance my effectiveness on the job.
5. Using [this product] would make it easier to do my job.
6. I would find [this product] useful in my job.
7. Learning to operate [this product] would be easy for me.
8. I would find it easy to get [this product] to do what I want it to do.
9. My interaction with [this product] would be clear and understandable.
10. I would find [this product] to be flexible to interact with.
11. It would be easy for me to become skillful at using [this product].
12. I would find [this product] easy to use.

A 7-point Likert-like scale would be better in getting more precise feedback for quantification but, seeing as there are already quite a few questions that the users would have

to answer, a 5-point Likert-like scale would be better. The seven-point Likert items can give more detailed insights, but it might take more time to analyze, especially in big surveys or when there's limited time. On the other hand, the 5-point scale is a simpler option that can be more efficient for capturing general sentiments("5-point vs 7-point Likert scale: Choosing the Best", 2024).

On the 5-point scale, the options are ordered from negative to positive and left to right respectively. Jim Lewis found significantly more response errors when the "extremely agree" and "extremely likely" labels were placed on the left. Jim recommended the more familiar agreement scale (with extremely disagree on the left and extremely agree on the right) (Lewis, 2019).

4.2: Consolidation of Contextual Inquiry Data

Affinity diagramming is the method I use to consolidate my findings. The affinity diagram process is good for making sense of findings gathered during research, as well as when you want to sort out ideas from ideation sessions (Dam & Teo, 2022).

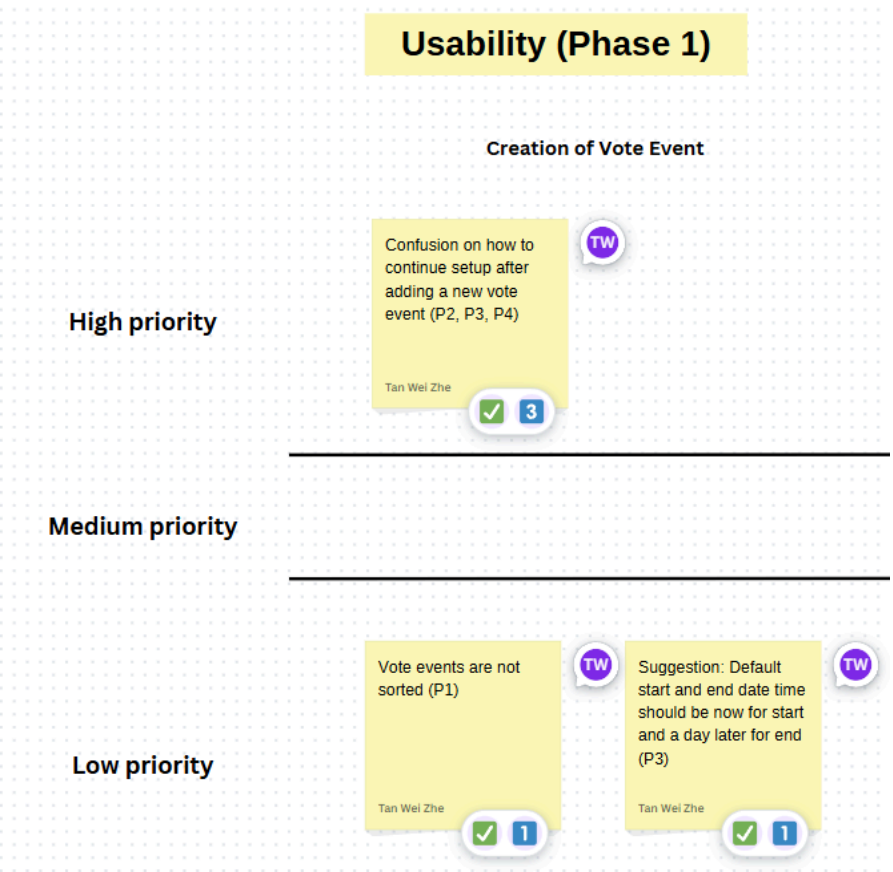


Figure 15: Portion of the affinity diagram (medium priority shrunk to decrease size)

Canva was used to create the affinity diagram. I created a post-it note for each issue or suggestion raised by each participant. If multiple participants mentioned the same issue, then I would combine them into a single post-it and indicate all participants who mentioned it with their anonymous identifier. The post-its are categorized by the column to the 5 voting stages that they are related to and by the row to their priority. I would then add a comment for each post-it, stating the recommended actions. Canva did not have any tagging system for the post-its, so I made use of the reaction functionality instead. I would react to each post-it with a number from 0 to 3 indicating the priority with 3 being the highest and 0 being no priority. Once the issue in the post-it is resolved, I would react with a tick.

Each post-it is **prioritized based on the severity of the problem**. Following what Nielsen stated, the severity of a usability problem is a combination of three factors: (1) The frequency with which the problem occurs: Did multiple participants encounter it? (2) The impact of the problem if it occurs: What would happen if the problem is not solved? (3) The persistence of the problem: Will it only affect first-time users? (Nielsen, 1994). The priorities allow me to better allocate my time to each issue, solving the more critical problems first.

4.3: Evolution of User Study Workflow

As this was the first time I conducted a user study, the workflow was far from perfect, but I did make changes over time to improve the overall process. These changes were mainly done over phase 1 of the user study.

1. **Expanded Definitions and Context:** More definitions and context have been added for the participants. For example, what is a vote event or what is a candidate?
2. **Pre-Task Data Awareness:** Participants are made aware of the relevant existing data in the system before starting their tasks
3. **Printed Task List:** For the first few participants, I gave out the instructions for each of the tasks they had to complete verbally. This was not very ideal as what I said could have some variations between participants, and I would also like to minimize my interaction with the participants when they are doing a task. I decided to type out a fixed task list that every participant will take reference from to complete their tasks. This list ensures every participant gets the same instructions.

4. **Enhanced Task Instructions:** Instructions in the task list have been made to be more well-defined. For example, some participants would ask what project IDs they should vote for. In this case, what exactly they vote for is not important since it is all fake data in the database. The instructions were updated to indicate which projects they would vote for, reducing ambiguity.
5. **Voice recording:** There was no voice recording done initially, and I had to manually type out the feedback after each task. This increased the time taken to complete the user study as I needed to use my laptop to record the feedback. Voice recording allowed me to directly converse with the participants without any interruption. It also serves as concrete evidence of what each participant mentions. The voice recording was done with the help of Zoom which also has a feature to transcribe the recording for easier analysis.

4.4: Contextual Inquiry

In this section, I will present the findings from the user study conducted with regard to the contextual inquiry.

4.4.1: Phase 1

Phase 1 was conducted on version 2 of the system, with a total of 4 participants. Since this is the first phase, all participants were new to the system. Below are the key findings from phase 1. They have the highest priority among the issues raised.

Finding 1: Confusion on how to continue setup after adding a new vote event

Referencing, figure 3 (3.1.2: Voting Process Stages), a vote event can be added by clicking on the new vote event button, which opens a modal that requires title, start time, and end time to add a vote event. After adding, the vote event will appear in the table. The confusion happened in the step after participants added a vote event, where participants were required to configure the voters for the vote event but they were unsure of how to navigate there. At this point in time, the manage button was labeled as ‘edit’ instead, and actions were not separated into voter and admin yet. They were supposed to click on the edit button to bring them to the manage vote event page (edit vote event page previously) which contains everything they need. They simply did not know such a page existed.

This was classified as a high-priority issue. The persistence of this issue is not high because this is more of a one-time off navigation issue that first-time users face. However, the impact is high because the vote event cannot be set up properly if users are unsure of where to navigate or what else is needed to set up a vote event. Furthermore, the frequency of this issue is high, with 3 out of the 4 participants encountering it.

Solution

One possible solution is to have a very clear description informing users on how and where to go to properly set up a vote event, but there might be users who will ignore such walls of text. The better solution I went with is to have the system automatically navigate to the managed vote event page upon the creation of the vote event. This instantly allows users to see what other stuff they can configure for the vote event.

Finding 2: Unsure what the candidate display options refer to

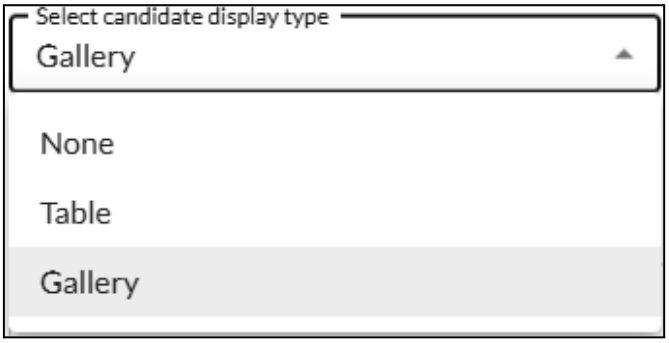


Figure 16: Candidate display dropdown selection in vote config modal

When setting the vote config for a vote event, there is a candidate display dropdown selection field where participants can select one of the three candidate displays (None, Table, Gallery). The feedback received from the participants was that they were unsure what each of the options meant. This was because no details were provided for the displays.

This was classified as a high-priority issue. The persistence of this issue is low because users will eventually understand what each option does after testing it out. The impact is high because first-time users will not be able to know what to choose for their vote events and cannot deliver what they want. The frequency of this issue is also high, with 3 out of the 4 participants encountering it.

Solution

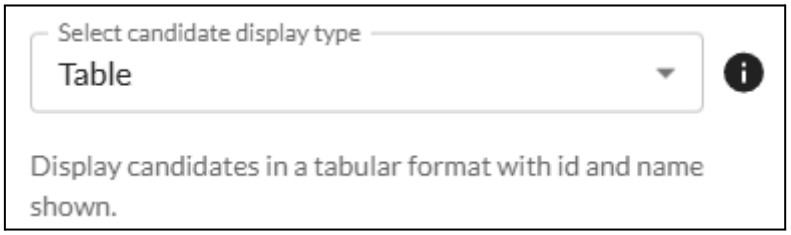


Figure 17: Updated candidates display selection

The first solution I thought of is to add a description for each of the candidate display options at the top of the vote config modal. However, keeping in mind that the system is customizable and that more options for candidate displays can be easily added, this wall of text at the top of the modal can grow indefinitely. At a certain point, it would be an inconvenience for users to read and navigate all the descriptions, the modal would also become too cluttered.

An improvement to this solution is to have the description appear below, after selecting a candidate display option. This allows users to understand what they have selected and it can scale properly with the number of options. Additionally, an info icon with a tooltip is added beside the dropdown to explain what candidate displays are in general and that users have to select the option to see its description.

Finding 3: Vote cast number not being noticed

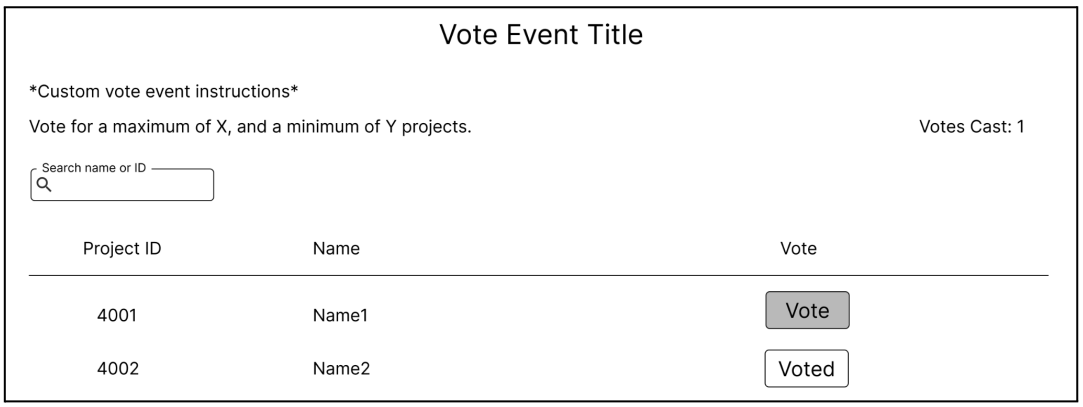


Figure 18: Old voting page design

On the voting page, there was a votes cast number at the top right of the page which changes based on how many candidates you voted for. However, where there are a lot of candidates, voters will have to scroll the page to view all the candidates, and the vote cast number would be scrolled out of view. I added it so that voters can keep a count of their votes, but my observations from the user study were that it would normally scroll out of view and the participants would not notice it.

This was classified as a medium-priority issue. The frequency of this issue is high with 3 participants not noticing it. The persistence is medium because it is possible to continue to not notice it beyond the first usage. However, the impact is low because there is a confirmation modal that indicates how many votes were cast before submission but it would still be good to let voters know how many votes they have cast for especially when they are scrolling and selecting from a long list.

Solution

Figure 19: New submit votes button (below and above minimum votes respectively)

The solution is to have the number of votes cast follow the user's view. Since the submit button is already following the user's view, it would be better to combine them. I would also like to further improve it by letting it indicate the minimum and maximum votes in some way. Although the minimum and maximum votes are indicated at the top of the page, having them follow the user's view can help to remind them. When the number of votes cast is below the minimum, the button will display the fraction of votes cast out of the maximum votes, with the minimum votes required in brackets beside. When the number of votes cast is above the maximum, the button will display the fraction of votes cast out of the maximum votes with the maximum votes allowed in brackets beside.

4.4.2: Phase 2

Phase 2 was conducted on version 3 of the system, which has some enhancements, including the improvements made from the issues in phase 1. A total of 5 participants were involved, with a mix of 3 returning and 2 new participants. A mix of participants allows for a wider variety of feedback as returning participants have seen the system and its previous issues, while new participants can give the first-time user experience. Below are the key findings of phase 2, with the highest priority out of all the findings. Overall, phase 2 has fewer issues raised.

Finding 1: The function of the edit button for vote events is not clear

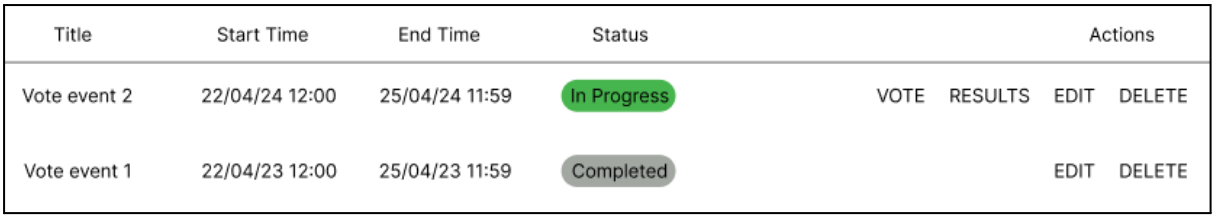


Figure 20: Old vote event table design

From the vote events page, participants were given the task of publishing the results of a vote event, which can be done through the results tab in the manage vote event page. However, they were unsure of how to navigate there through the vote event table. In the old design, the edit button leads to the edit vote event page (now referred to as the manage vote event page) which contains the results tab that they require. They hesitated clicking the edit button because they were unsure if it could do what they needed, some thought that the edit button only edits the title and timings. To add on there is a possibility of the results button adding to the confusion as publishing results might be located in there. The results button is only present when results are published, and it is what voters will use to view the customized results.

This is a high-priority issue. The frequency of this issue is high, with all participants having issues with the edit button. The persistence is low because this is a first-time user issue only. The impact is high because it is important to know that all the configurations can be accessed through the edit button.

Solution

The change is reflected in figure 3 (3.1.2: Voting Process Stages). Firstly, the edit button was renamed to manage to better reflect its action (as suggested by some participants). Secondly, to improve the clarity of the results button, it will be renamed to view results, which also fits the fact that the buttons are all verbs. The buttons will be split into two columns, a voter actions column with vote and view results, and an admin actions column with manage and delete, which is only visible to administrators. This separation better indicates the usage of each button.

Finding 2: Participants could not locate where to set the period for internal voter registration

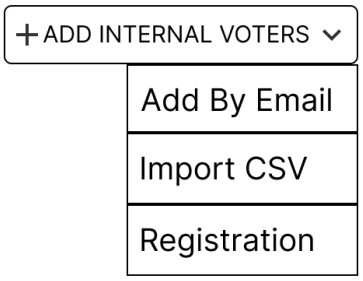


Figure 21: Old add internal voter menu design

Administrators could set a period for internal voter registration by accessing it from the registration button found in the add internal voter menu. It was designed this way because registration allows users to register for a vote event, which adds them as an internal voter of the vote event when they do so. Being one of the methods to add internal voters, it was placed inside the add internal voter menu. However, from the point of view of the participants in the user study, the link between registration and adding internal voters is not clear, which is why they did not access the add internal voter menu and had trouble finding where to set a period for registration. What they do know is that it should be somewhere in the voter management tab, which was observed by them searching inside of it.

Solution

For the sake of usability, it is better to move the registration button out of the add internal voter menu because the logical link between adding internal voters and registration is not straightforward. This can be seen in figure 4 (3.1.2: Voting Process Stages).

4.4.3: Phase 3

Phase 3 was conducted on version 4 of the system, with the same workflow as phase 2. Phase 3 was initially conducted with 2 returning participants who both participated in phases 1 and 2. The two participants were able to complete the tasks swiftly in phase 3, with no big issues being identified, except for an issue with the tooltip placement on the voting card. Overall, The feedback from the two returning participants was limited, likely because they had **become quite familiar with the system**, having completed the same tasks multiple times.

It wasn't good to just conclude the user study like that so I got an additional participant who was new to the study, to gather fresh insights. For this session, I paid particular attention to areas where **issues had been identified previously**, observing if the new participant encountered any challenges and asking for feedback on those aspects. Overall, the participant found the changes intuitive and had a positive experience using the system. However, the participant did raise one issue. They felt that the submit votes button (figure 19) looked more like a static box that dynamically tracked the number of votes instead of a button. This is something that the returning participants might have overlooked because they already expect a submit votes button in that position.

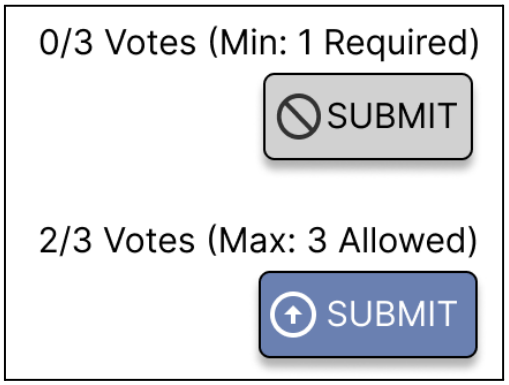


Figure 22: Proposed changes to submit votes button

There are no plans to release version 5 of the system during the duration of this project because phase 3 was conducted right at the end. However, version 5 could look into making the submit votes button more visually distinct to ensure that it more clearly resembles a button. After taking a closer look at the button I feel that it might be because there is too much text inside it, making it look like some information display. As shown in figure 22, version 5 could potentially have the vote count details outside of the button, and have icons to

better represent as an actionable item. Another reason is that there are buttons to vote for each candidate on the page, and since the submit votes button is in a floating position, it should have a more defined shadow as compared to other buttons to show that it is floating.

4.5: Survey

The survey has a total of 9 responses, with 4 from phase 1 and 5 from phase 2. 3 of the participants participated in both phases and have responded twice in the survey. The survey was not conducted in phase 3 as there were no changes to the customization features.

4.5.1: Data Processing

To quantify the data, ratings from strongly disagree to strongly agree are mapped to numbers 1 to 5 respectively. For each participant and feature, the ratings for the usefulness and ease of use questions are then aggregated into single ratings for perceived usefulness and perceived ease of use, rounded to two decimal places for some level of precision. This was done after exporting the Google Forms data to Google Sheets. There were also questions that asked participants to state any redundant customizations, which were left blank by all of them.

4.5.2: Final Data

Candidate displays

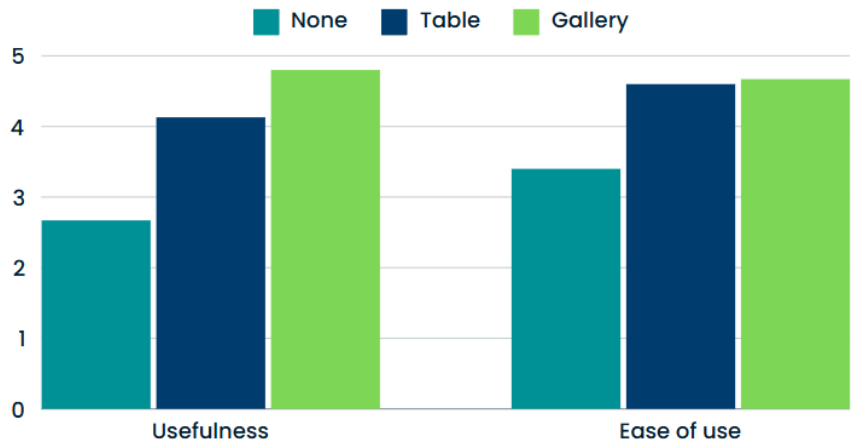


Figure 23: Bar graph of usability and ease of use ratings for the three candidate displays for phase 2

When it comes to the different candidate displays, no candidate display has the lowest ratings for usefulness and ease of use, while the gallery view candidate display has the highest, with

the table view candidate display being in the middle. What this shows is that **participants prefer having more information and visuals when voting**. The preference for more information suggests that visual aids, like poster images and candidate details, enhance the decision-making process, as participants can quickly assess their options and feel more confident in their choices.

However, it's important to note the conditions under which this user study was conducted. Both the client and server were running **locally on the same machine**, meaning network latency, which could impact loading times for candidates and their images, was not factored into the participants' experience. In a real-world setting, such as during the Splashdown poster presentations, network delays may affect the user experience, especially for more data-heavy displays like the gallery view. The absence of this factor in the study might have contributed to the consistently high ratings for the gallery and table views, as participants did not experience any of the potential lag that could occur when using these views remotely.

Additionally, the study **did not simulate the actual poster presentations that occur during Splashdown**, which might have further impacted how participants perceived the different candidate displays. In a real event, participants might rely more on external information from the poster presentations, possibly reducing their need for detailed candidate displays in the voting interface. This could explain why the 'no candidate display' option scored poorly, as participants in the study were evaluating the displays in isolation, without any external context.

Role Weights and Results Customization

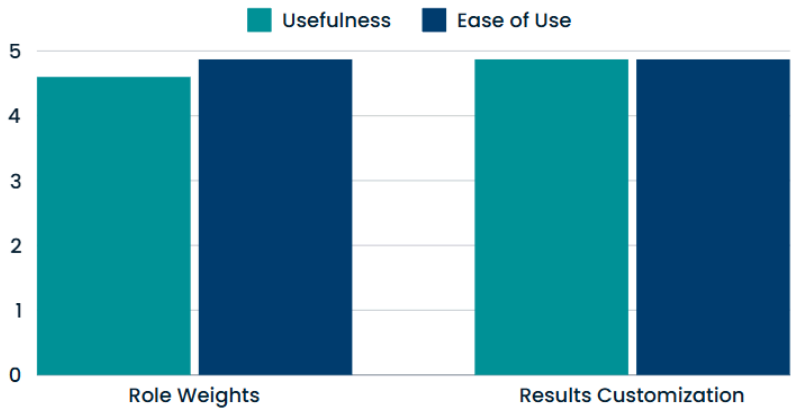


Figure 24: Bar graph of usability and ease of use ratings for role weights and results customization for phase 2

These two customization features received very high ratings, indicating that participants greatly value customization in these areas of the voting process. Role weights allow administrators to ensure that votes reflect proper representation across different voter groups, which adds fairness and accuracy to the results.

Similarly, the ability to customize results was seen as highly useful and easy to use. The high score for these features suggests that participants value having control over both the voting process and how results are communicated. According to TAM, perceived usefulness and perceived ease of use determine the user's behavioral intention to use, which means that the high ratings imply that these two customization features are something that they will accept and use. However, the downside of TAM is that there is no true benchmark across different systems. This makes it hard to do a comparison with existing systems. Nevertheless, compared to the individual candidate displays, these two customizations were more valuable to the participants.

Comparison between phase 1 and phase 2

The ratings from the three participants who participated in both phases 1 and 2 were taken and compared to see if there was any improvement from phase 1 to phase 2.

Generally, usefulness ratings of the customization features were better in phase 2 after changes were made to the system. This shows that the changes made to the features made the participants find them more useful. However, it could also be the case that participants had a better understanding of the usefulness of the feature after using it for the second time.

Ease of use, on the other hand, did mostly remain the same for the features. Participants might have found the interfaces intuitive from the start, or the change was not sufficient to increase the ratings to the next tier. However, the consistency in the ratings shows that the changes made did not make the use of the features more complex.

Chapter 5: Good Software Engineering Practices

This project is ultimately a software engineering project, so it is important to follow good software engineering practices throughout the project. This chapter covers the important software engineering practices applied in the project, to develop a system of good quality.

5.1: Software Development Lifecycle

The chosen software development lifecycle (SDLC) model was the **iterative model**. Iterative development allows me to produce different versions of the system incrementally. Not only does it provide more **flexibility and adaptability to changes**, but it also allows me to have **early-stage user feedback on the actual system** and make the required changes for subsequent versions (Awati, 2023). The iterative development was done in a **breadth-first approach**, where a complete but basic version of features for each stage of the voting process was created before further enhancing them in future versions. This approach also ensured a comprehensive system could be tested as a whole while new features were incorporated gradually, maintaining a focus on quality and stability throughout the development process.

The development process was organized into three Github milestones. The full milestone details are in the appendix (A - Milestone Details). Milestones 1 and 2 have 1 week of buffer at the end, which was designed to prevent the snowballing of deadlines.

Milestone 1: Core system development (Version 1)
Milestone 1 focused on delivering a fully functional, base version of the voting system, covering all stages of the voting process. The goal was to establish a foundational system capable of handling the end-to-end voting workflow, from creating vote events to managing basic voting configurations and consolidating results.

Milestone 2: Enhanced customization and user testing (Version 2)
Milestone 2 built upon the core system by adding more customizable options and features. The first phase of the user study was also conducted toward the end of the milestone. The goal of phase 1 was not only to gather feedback to refine the system but also to fine-tune the user study process, ensuring clarity and reliability for future phases.

Milestone 3: User feedback integration and finalization (Versions 3 and 4)

Milestone 3 involved incorporating the user feedback from phase 1 of the user study into the system to create version 3, which was used for phase 2 of the user study. The system will then be further refined with the feedback from phase 2 to produce version 4 which is the final version. Phase 3 of the user study will do a final check on version 4 as a final form of evaluation.

Sprints

Each milestone consisted of several 2-week sprints, organized to incrementally build and test features. In general, each sprint included planning, design, development, testing, and documentation phases, ensuring that every new feature was fully validated and documented before proceeding to the next sprint. A sprint starts with planning, where the tasks required are generated. Most of the time, there would already be tasks generated beforehand as I had already planned out the outline for every sprint before each milestone, so the planning task would add or refine tasks. Next comes the design phase, where design details are written out, and design diagrams and mockups are created. User stories are also created for important features (appendix B - User Stories). Implementation begins once all these are done, followed by testing, where all the required test cases are added. The sprint will end with more documentation before finally closing with a reflection. There are sprints where other activities like conducting user studies are carried out, but more or less, there will always be planning, documentation, and reflection for every sprint, and design and testing for every implementation. All of which are important stages of software engineering.

By organizing the development into these structured phases and combining it with a strong testing strategy and iterative feedback loop, this approach ensured that each milestone delivered a well-defined, thoroughly tested set of features that could reliably support the voting system.

5.2: Project Management

Good project management is what really ties everything together, and GitHub projects is the tool I utilized for this. GitHub Project offers a variety of features for me to have **proper planning and tracking of my project**. I mainly utilized 3 different views for my project.

Kanban board view

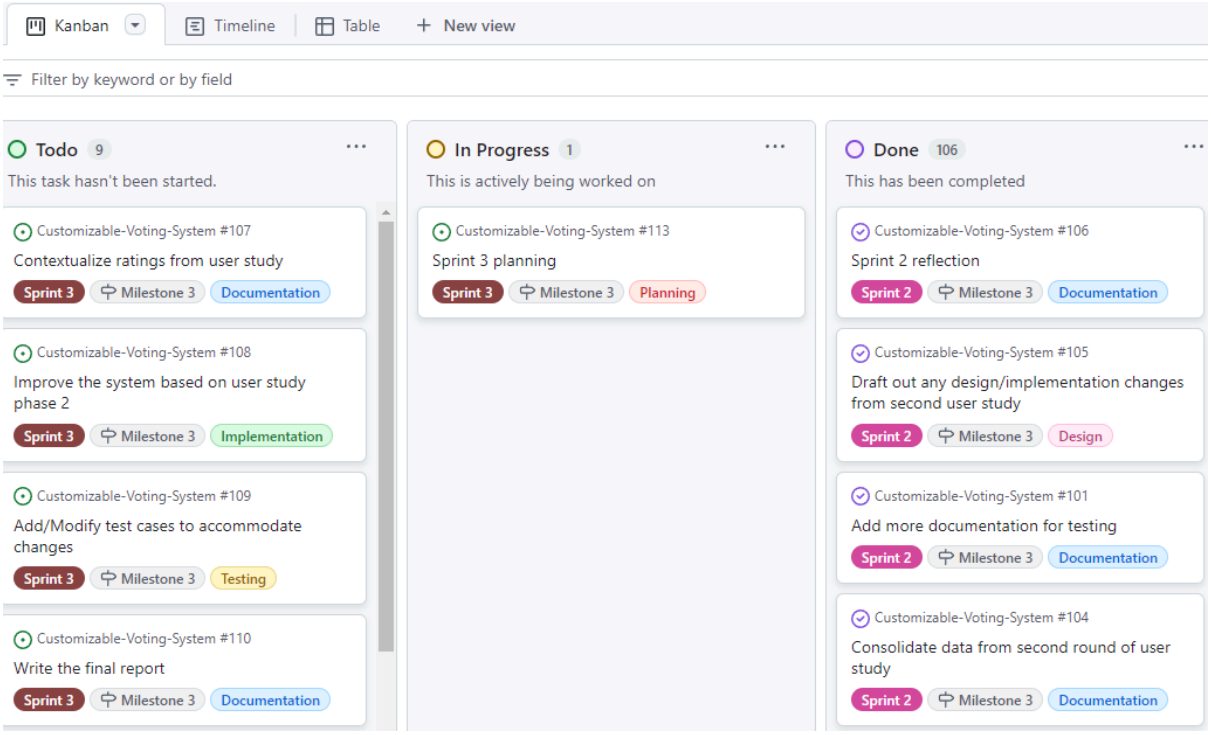


Figure 25: Kanban board view of tasks (3/5 columns shown)

The Kanban board view is the one I use the most frequently. This is where I will create and change the status of each task. There are 5 columns: (1) Todo column, with tasks that will be worked on in the future. (2) In Progress column, with the task that I currently am working on, (3) Done column, with all the completed tasks. (4) Meetings column, with the meeting dates and details of my supervisor meetings. (5) Links column, with important links for easy access.

Table view

Filter by keyword or by field

	Title	Type	Start Date	Labels	Milestone
57	Smoke testing for milestone 1 #53	Testing	Jul 29, 2024	buffer	Milestone 1
58	Finish up missing documentation for milestone 1 #54	Documentation	Jul 30, 2024	buffer	Milestone 1
59	Milestone 1 reflection #55	Documentation	Aug 4, 2024	buffer	Milestone 1
60	Draft our design for sprint 1 features #57	Design	Aug 5, 2024	Sprint 1	Milestone 2
61	Generate user stories for sprint 1 features #56	Planning	Aug 5, 2024	Sprint 1	Milestone 2
62	Sprint 1 planning #118	Planning	Aug 5, 2024	Sprint 1	Milestone 2
63	Add ability to copy voters from another event #58	Implementation	Aug 7, 2024	Sprint 1	Milestone 2
64	Add ability to import voters from csv #59	Implementation	Aug 8, 2024	Sprint 1	Milestone 2
65	Add registration for internal voters #61	Implementation	Aug 9, 2024	Sprint 1	Milestone 2
66	Add ability to automatically generate external voter IDs #60	Implementation	Aug 9, 2024	Sprint 1	Milestone 2
67	Add authentication for external voters #62	Implementation	Aug 10, 2024	Sprint 1	Milestone 2
68	Add access control for voters #63	Implementation	Aug 11, 2024	Sprint 1	Milestone 2
69	Sprint 1 frontend component testing #64	Testing	Aug 13, 2024	Sprint 1	Milestone 2
70	Sprint 1 backend unit and integration testing #65	Testing	Aug 14, 2024	Sprint 1	Milestone 2
71	End to end testing for sprint 1 features #66	Testing	Aug 15, 2024	Sprint 1	Milestone 2
72	Documentation and reflection for sprint 1 #67	Documentation	Aug 17, 2024	Sprint 1	Milestone 2
73	Draft out design for sprint 2 features #68	Design	Aug 19, 2024	Sprint 2	Milestone 2

Figure 26: Table view of tasks (status and end date fields hidden to reduce size)

The table view allows me to see a very compact view of all my tasks. It has all the columns for each task which I can group, sort, or filter by. I can easily manage the different information of each task from this view.

Timeline view

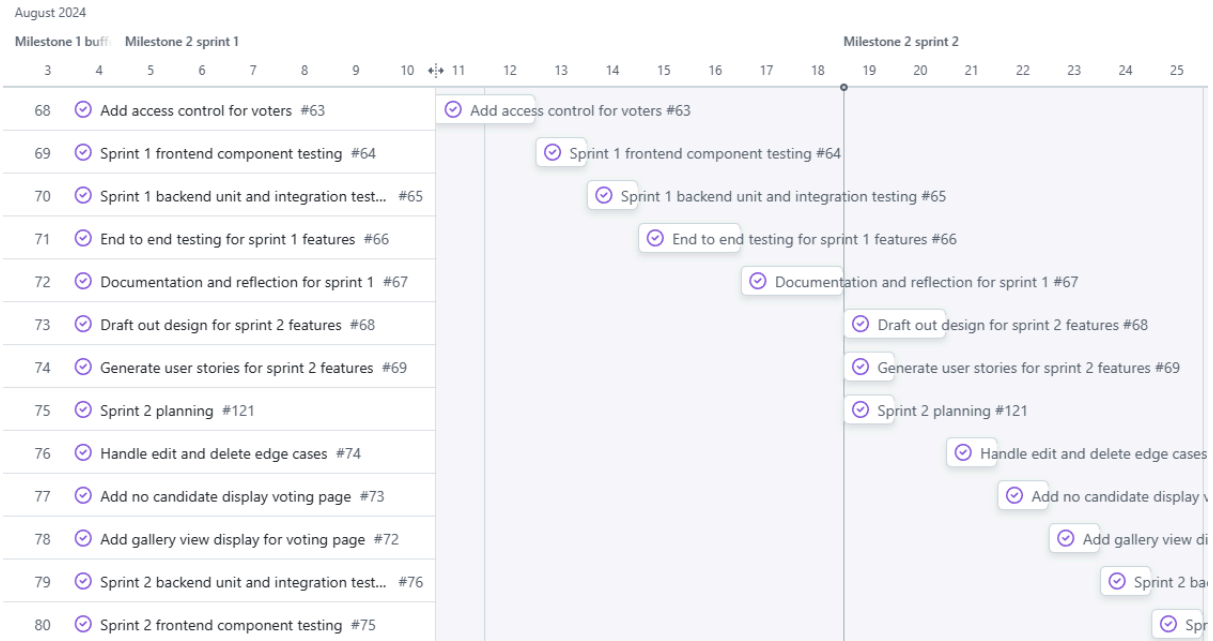


Figure 27: Timeline view of tasks

The timeline view allows me to put tasks in chronological order. I can see the flow of tasks over a specific timespan and drag and drop tasks based on their start and end dates. Vertical line markings are added to denote the different sprints for easier reference.

Workflow

Before each sprint, the necessary tasks are created in the To-Do column of the Kanban board. Each task is converted into a GitHub issue and assigned the appropriate milestone, label, and type. If a task name lacks clarity, additional details are included in the description. Tasks are moved to the In Progress column when work begins and to the Done column upon completion. The workflow is configured such that GitHub issues automatically close when moved to the Done column. Afterward, tasks are adjusted in the timeline view to reflect their correct start and end dates.

Insights

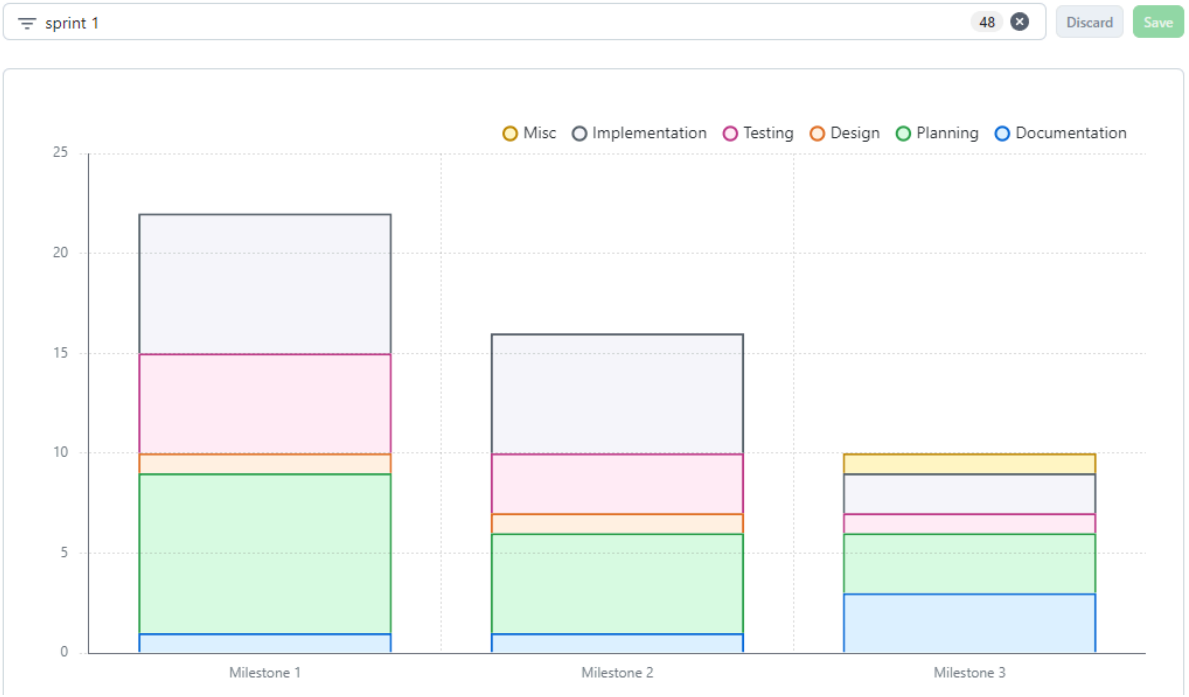


Figure 28: Chart generated by GitHub Project's insights

GitHub Projects also features an insights tool that allows for the creation of customizable charts based on project data. For example, a chart was generated to analyze the types of tasks created for sprint 1 across milestones. Sprint 1 of milestone 3 focused on conducting phase 2 of the user study, which explains the reduction in implementation and testing tasks but an

increase in documentation tasks. In contrast, milestone 1 contained a larger number of tasks due to the implementation of many basic features.

Limitations

This was the first time that I used GitHub projects for project management, and I found the learning curve to be quite steep. The interface is complex, with various views and features, and it took time to discover and utilize many of them, leaving some features potentially undiscovered. Additionally, each task must be linked to a GitHub issue, as the tool primarily manages issues and pull requests, making it less user-friendly for general task management compared to other software. Another limitation is the lack of automatic tracking of start and end dates when moving tasks through the Kanban columns; this must be done manually in the timeline view. Lastly, while fields for start and end dates exist, the insights feature does not provide a way to track the time spent on each task.

5.3: Implementation

The implementation of the system follows best practices in software development. Aspects like code reuse, coding standards, and effective project organization to ensure readability and maintainability across both frontend and backend components. There are also other important details like UI libraries, responsive design, and code linting and formatting that contribute to the overall quality of the system.

5.3.1: Reusable Components

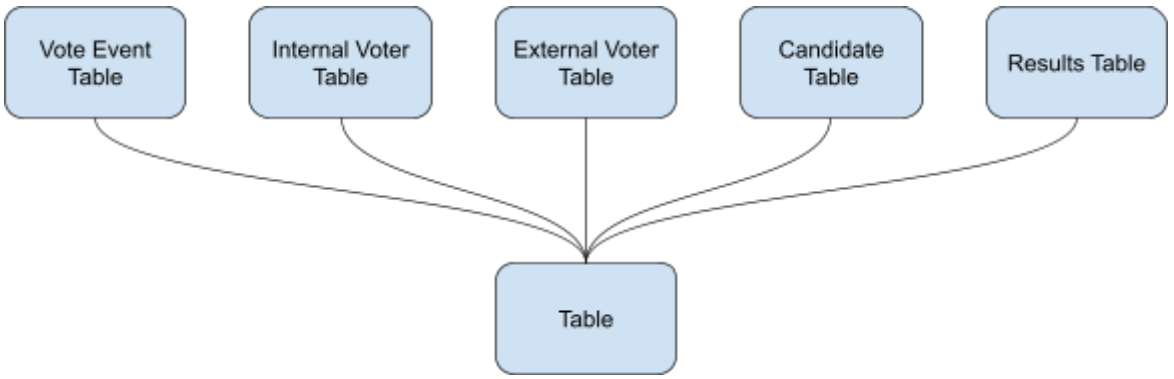


Figure 29: Specific table components compose of the same table component

Creating reusable components is important, it reduces repeated code in components that have a similar structure or logic. In the example above, the table component contains all the code

related to the common table elements, and more specific tables like the results table are created by composing this table component. This is also a good example of how OCP is present at the core of how components are designed, where the functionality of the table component is extended by creating new components that compose it without directly modifying its code.

The concept of having a common component that more specific components are built from is present in not just the tables example but in many other common UI elements like modals, menus, text fields, etc. Generally, when two components have similar code, a common component can be abstracted out, which both the components will then compose. It allows me to have a scalable and maintainable codebase.

5.3.2: Coding Standards

I followed the coding standards of the previous developers that were already present in the codebase. On top of that, I also implemented the following coding standards to further improve the quality of written code.

Guard Clauses: Keeps the happy path visible by handling edge cases early.
Conditional Rendering with &&: For readability, it is better to use && to conditionally render components, as it avoids deep nesting.
Optional Chaining: The optional chaining operator (?.) is a neat way to check for undefined values without the need for an if else statement.

5.3.3: Linting and Formatting

There are tools that help to ensure that code quality is of a certain standard throughout the entire codebase.

Code Linter: Eslint is configured to enforce a consistent code style by identifying and reporting patterns that may lead to errors.
Code Formatter: Prettier is used to format the code. It enforces a standardized style across all code written by developers.

Pre-commit Git Hook with Husky

A pre-commit Git hook is configured using Husky to automatically type check, lint check, and format check all the code before a commit. Any errors or warnings that are thrown during the checks will automatically put a stop to the commit process. This ensures that all committed code passes the type and style checks at a minimum.

5.3.4: Project Structure and Organization

Component-Based Folder Structure: Each component will have its own folder, to group all its resources. The component folders are further organized by their UI type (tables, modals, forms, etc).

Ordered Imports and Logical Blocks: Imports are sorted based on their paths at the top of the file, and logical blocks of code are separated by a blank line. This ensures proper organization of code in each file.

5.3.5: UI Library

Material UI (MUI) is the UI library that is used in the frontend. It includes a comprehensive collection of prebuilt components that are ready for use in production right out of the box and features a suite of customization options that make it easy to implement our own custom design system on top of the components (*Overview - Material UI*, n.d.). Using such a library really helps to speed up development as it provides many useful components out of the box. Most of the components in the frontend are composed of MUI components.

On top of that, MUI implements Google's Material Design, following their best practices of UI design. This means that the components offered have a good standard of quality and functionality, which contributes to the overall build quality of the voting system.

5.3.6: Mobile Responsiveness

Splashdown is usually held in person. This means that voters would most likely be using their mobile devices to vote during the voting event, as it is more convenient. To ensure that the system is responsive to screen sizes, I made use of MUI's responsive Grid and Box components, which can automatically adjust content based on screen size. Additionally, MUI's breakpoint feature allows me to implement different styles at specific screen size

breakpoints. For non-MUI components, I made use of the useMediaQuery hook to access the different breakpoints for styling.

5.4: Testing

Testing helps to ensure that the system is functioning correctly. It is an important stage in the development of any software. This section will cover the different forms of testing I did, which provided me with a high confidence in the reliability and functionality of the system.

5.4.1: Frontend Testing

Frontend testing verifies that the user interface displays and functions correctly, and I used Cypress to conduct end-to-end (E2E) and component testing in the frontend. E2E tests ensure all parts of the system (frontend, backend, database) integrate and work properly together. Component tests evaluate individual components in isolation, allowing for early bug identification. Each component is tested by mounting it by itself, utilizing spies to mock external functions, and intercepting API calls to isolate tests from the backend. In terms of what to test for in the frontend, I followed a quote by Kent C. Dodds. “The more your tests resemble the way your software is used, the more confidence they can give you.” (Dodds, 2019).

5.4.2: Backend Testing

The backend is tested using Jest and Supertest for unit and integration testing. Unit tests are done first to ensure that route, helper, and model functions in the three layers perform their intended functionality. Next, a bottom-up approach is used for integration testing to verify the interaction between the layers. The integration goes from the model layer all the way up to the route layer. This ensures everything in the backend is functioning correctly, and can properly serve API requests.

5.4.3: Smoke Testing

Smoke testing is done as a sanity check on the critical functions of the system. Successfully passing smoke tests helps to build confidence that the system has achieved a certain level of quality and stability (Sheremeta, 2024). I first use the set of user stories to test the system, before going for a more exploratory style of testing after. Exploratory testing is a powerful software testing method that allows testers to adapt and experiment dynamically, adjusting

their approach based on real-time observations of the system (Sachedina, 2024). It allows me to explore and potentially discover bugs not covered by other testing. Smoke testing was conducted twice in total, one after version 1, and another after version 4 of the system.

5.4.4: Regression Testing and Continuous Integration (CI)

Regression testing ensures that code changes do not break any of the existing functionalities. This should be done whenever changes are introduced to the system. To make regression testing automated, I added the running of the test cases to the CI pipeline for both the frontend and backend. The tool used for this was Github Actions. Future code changes will undergo automatic regression testing, ensuring that our system remains functional.

5.4.5: Code Coverage

The backend has 308 test cases with a line coverage of 86.32% for model functions, 95.97% for helper functions, and 99.54% for routes. The frontend has 292 test cases with 86.78% line coverage for components. 100% coverage is ideal, but testing can never guarantee that the code is bug-free (except for formal testing methods). It is important for me to properly allocate my time and resources across all stages of the software engineering process.

Chapter 6: Conclusion

6.1: Summary

This report outlines the design, development, and testing of a customizable voting system for Skylab v2, aligning closely with the project objectives of identifying customizable features, designing for customization, and improving and evaluating based on user feedback.

The first objective, identifying customizable features, was achieved by focusing on key areas such as candidate display, role weights, and results customization. These features were adapted from existing systems and literature and were designed to best fit Skylab v2. We did not simply copy the features directly but utilized the context of Orbital and Skylab v2's existing capabilities to optimize the features into ones suitable for our system.

The second objective, designing for customizability, was achieved by first taking a modular approach, grouping customization into the different voting stages. The frontend was designed with component-driven principles to isolate customizable parts, a config-based factory pattern to scale options, and centralized mapping of customizations to maintain organization. These designs were further complemented by the use of progressive disclosure to control the amount of information presented to the user. The database was designed to logically organize voting event data across stages, ensuring both flexibility and scalability in how customizations are applied. All these elements work together to create a flexible and scalable customizable voting system.

The third objective, improving usability and evaluating primary customization through user studies, was achieved through iterative user testing. Multiple phases with a mix of new and returning users provided valuable insights into areas of improvement via contextual inquiry. The survey, utilizing the Technology Acceptance Model (TAM) framework, helped to evaluate the acceptance of the primary customization features, providing a clear measure of their perceived usefulness and ease of use.

The entire process is complemented by having good software engineering practices throughout the development process. With proper project management, a carefully planned

software development lifecycle, good implementation practices, and comprehensive testing, we ensured that the system is of reasonable quality.

6.2: Limitations

While I managed to create a customizable system, there are certain limitations. First, the customization options, such as the different candidate displays, **provide flexibility but cannot be customized in depth**. For example, users can choose from various predefined display types, but they are unable to create their own layouts from scratch. For any new display type, development is required before it can be added as an option. However, once developed, it can be easily added with the help of the factory pattern.

Secondly, there are challenges with ensuring **consistency across components generated by factories**. For example, the add-internal voter options are components generated by a factory, and since components in React are UI elements, they have to be homogeneous to fit nicely in the UI. You cannot have a factory generate different components with different UI shapes and expect them to fit nicely in the same spot. The add-internal voter options are wrapped inside modals, which makes them homogeneous.

Lastly, my observations from the user study were that the task-oriented process results in participants **solely focusing on completing their task**, which sometimes prevents them from noticing additional details or fully exploring the system's capabilities. This tunnel visioning means minor interface elements might be overlooked, which could result in less feedback on the overall user experience. This can be countered by a more exploratory approach by letting participants "play" around with the system for a set period of time.

6.3: Future Work

Moving forward, several enhancements can be made to the system. First, the **issues identified during phase 3 of the user study**, in particular, the appearance of the submit votes button, should be addressed to ensure a more intuitive user experience. Redesigning the submit votes button to make it appear distinctly interactive, for example, would help eliminate any potential confusion for users. It does not stop here. The system can continue to iteratively undergo more user studies of varying styles to gain deeper insights and improve.

Secondly, the system could benefit from **expanded voter and candidate customization**. By allowing for additional configuration, such as custom roles or permissions for voters and more detailed candidate profiles, future versions could accommodate a wider range of voting scenarios. This flexibility would make the system adaptable for various voting events.

Lastly, offering **deeper customization options for candidate displays** would allow users to create highly tailored interfaces. Currently, displays can be chosen from a set of predefined layouts, but an advanced customization feature that lets administrators arrange display elements and adjust styling would provide even greater control. This enhancement would allow users to create displays that more closely align with their event requirements.

References

5-point vs 7-point Likert scale: Choosing the Best. (2024, February 20). QuestionPro. https://www.questionpro.com/blog/5-point-vs-7-point-likert-scale/#:~:text=The%20seven%2Dpoint%20Likert%20items,efficient%20for%20capturing%20general%20sentiments.

Amaresan, S. (2020, May 5). The Expert's Guide to Contextual Inquiry interviews. HubSpot Blog. https://blog.hubspot.com/service/contextual-inquiry

Awati, R. (2023, June 13). iterative development. Software Quality. https://www.techtarget.com/searchsoftwarequality/definition/iterative-development#:~:text=Iterative%20development%20can%20eliminate%20many,changes%20to%20improve%20the%20product

Barnes, T.D., & Rangel, G. (2014). Election Law Reform in Chile: The Implementation of Automatic Registration and Voluntary Voting. Election Law Journal, 13, 570-582.

Bhimani, K. (2023, September 15). Composing components in react applications. https://www.dhiwise.com/post/composing-components-in-react-building-blocks-for-apps

Burden, B.C., & Neiheisel, J.R. (2013). Election Administration and the Pure Effect of Voter Registration on Turnout. Political Research Quarterly, 66, 77 - 90.

Dam, R. F. and Teo, Y. S. (2022, May 2). Affinity Diagrams: How to Cluster Your Ideas and Reveal Insights. Interaction Design Foundation - IxDF. https://www.interaction-design.org/literature/article/affinity-diagrams-learn-how-to-cluster-and-bundle-ideas-and-facts

Davis, F. D. (1989). Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology. MIS Quarterly, 13(3), 319–340. https://doi.org/10.2307/249008

Dodds, K. C. (2019, May 24). Avoid the test user. https://kentcdodds.com/blog/avoid-the-test-user

Kurz, S., Maaser, N., & Napel, S. (2016). On the Democratic Weights of Nations. Journal of Political Economy, 125, 1599 - 1634.

Lewis, J. R. (2019, February 5). Essay: Is the Report of the Death of the Construct of Usability an Exaggeration? - JUX. JUX - The Journal of User Experience. https://uxpajournal.org/death-usability-exaggeration/

McGhee, E. (2019). Effects of Automatic Voter Registration in the United States.

Nielsen, J (1994, November 1). Severity Ratings for Usability Problems. https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/

Overview - Material UI. (n.d.). https://mui.com/material-ui/getting-started/

Quick start – react. (n.d.). https://react.dev/learn#components

Refactoring.Guru. (2024, January 1). Factory method. https://refactoring.guru/design-patterns/factory-method

Sachedina, F. (2024, April 1). What is Exploratory Testing? - The Ultimate Guide. https://www.globalapptesting.com/blog/what-is-exploratory-testing#why

Shah, E. (2024, April 30). Exploring the factory method design pattern - Eshika Shah - Medium. Medium. https://medium.com/@eshikashah2001/exploring-the-factory-method-design-pattern-4d270b6ff935

Sheremeta, O. (2024, August 29). What is Smoke Testing examples and when is it done? testomat.io. https://testomat.io/blog/what-is-smoke-testing-in-examples-and-when-is-it-done/

Street, A., Murray, T.A., Blitzer, J., & Patel, R. (2015). Estimating Voter Registration Deadline Effects with Web Search Data. Political Analysis, 23, 225 - 241.

Yusifov, F. (2018). Weighted Voting as a New Tool of Democratic Elections. IFAC-PapersOnLine, 51, 118-121. [Original source: https://studycrumb.com/alphabetizer]

Appendix

A - Milestone Details

In general, every sprint has a planning task at the start, documentation, and reflection tasks towards the end. There are also design tasks for the implementation to be done for the sprint. All of these are omitted in the details below to make it more concise.

Milestone 1: 11 weeks from 20/5/24 to 4/8/24 (5 x 2 week sprints + 1 week buffer) (Summer holidays)
Milestone 1 focuses on delivering a basic voting system that can complete every stage of the voting process. Each sprint will involve completing both the frontend UI and backend APIs for the listed items, and also basic unit and integration tests. The last sprint will involve end-to-end testing of the system. There will be design and planning done at the start of every sprint to properly design and document what is to be done during the sprint. Milestone 1 will produce version 1 of the system.

The voting process is separated into the following stages:
stage 1: Creation of vote event with basic details like name and duration (sprint 1)
stage 2: Adding voters to the vote event (sprint 1)
stage 3: Selecting candidates (sprint 2)
stage 4: Configuring voting style and display + Commencement of voting (sprint 3)
stage 5: Consolidation and release of results (sprint 4)

Sprint 1 (20/5/24 - 2/6/24)

- Creation and deletion of vote events
- General settings for vote events
- Voter management config
- Internal and external voter tables
- Default way of adding internal voters (by email) and external voters (voter ID)

Sprint 2 (3/6/24 - 16/6/24)

- Plan out user studies (submit to DERC for review)
- Candidates table in the candidates tab

- Basic way to add candidates by project ID
- Batch add candidates by selecting from specific categories (cohort year and achievement level)
- Add role-based access control

Sprint 3 (17/6/24 - 30/6/24)

- Vote event phases
- Vote config tab
- List view display voting page

Sprint 4 (1/7/24 - 14/7/24)

- Results tab in management of vote event
- Role weights customization
- Publishing of customized results
- Results page for vote event

Sprint 5 (15/7/24 - 28/7/24)

- Add request body validators to routes
- Set up end-to-end testing
- Conduct code review

1 week buffer (29/7/24 - 4/8/24)

- Conduct smoke testing
- Finish up missing documentation
- Milestone 1 reflection

Milestone 2: 7 weeks from 5/8/24 to 22/9/24 (3 x 2 week sprints + 1 week buffer) (week 0 to week 6)

Milestone 2 focuses on adding customization options on top of what is done in milestone 1. This involves more ways of adding voters for voter management, more voting page candidate displays, and also a search feature for the current tables. There will also be other enhancements like authentication, access control, and handling of edge cases when editing and deleting. All of this will contribute to version 2 of the system, which phase 1 of the user study will be conducted on, towards the end of the milestone.

Sprint 1 (5/8/24 - 18/8/24)

- Auto generation of external voter IDs
- Import CSV for internal and external voters
- Copy voters from another event
- Registration feature for internal voters
- Access control for vote events
- Authentication for external voters

Sprint 2 (19/8/24 - 1/9/24)

- Add automated testing to CI
- Handle all delete and edit edge cases
- No candidate display voting page
- Gallery view display voting page

Sprint 3 (2/9/24 - 15/9/24)

- Improve factory pattern
- search feature for tables
- User study phase 1

1 week buffer (16/9/24 - 22/9/24)

- Finish up user study phase 1
- Milestone 2 reflection

Milestone 3: 8 weeks from 23/9/24 to 17/11/24 (4 x 2 week sprints) (Recess week to week 13)

Milestone 3 focuses on consolidating the findings from phase 1 of the user study and making the appropriate changes before conducting phase 2. There will not be new features added in this milestone. On top of that, the website will also be made responsive to screen sizes. There are also documentation tasks like writing the final report. Besides adding tests for any new changes, there will also be a smoke test on Version 4 with all the user stories. Version 3 of the system is done after making the website responsive and also implementing changes from phase 1 of the user study. The improvements made from phase 2 of the user study will produce version 4, which phase 3 of the user study will be conducted on.

Sprint 1 (23/9/24 - 6/10/24)

- Make the website responsive to screen sizes, especially important for the voting/results page

- Consolidate user study phase 1 findings
- Make improvements according to phase 1 user study findings
- Add/Modify test cases based on the changes to the system
- Prepare for user study phase 2

Sprint 2 (7/10/24 - 20/10/24)

- Conduct user study phase 2
- Consolidate user study phase 2 findings
- Add documentation for user study phases 1 and 2
- Update how planning is documented
- Update documentation for testing

Sprint 3 (21/10/24 - 3/11/24)

- Make improvements according to phase 2 user study findings
- Add/Modify test cases based on the changes to the system
- Contextualize the ratings from the user study
- Conduct a code review
- Smoke test the system with the help of user stories
- Conduct user study phase 3
- Consolidate phase 3 findings
- Write final report

Sprint 4 (4/11/24 - 17/11/24)

- Write final report
- Submit final report (Week 12 Wed 6/11/24 deadline)
- Work on the final presentation

B - User Stories

As a/an…	**I want to…**	**So that I can…**
Administrator	Create a vote event	Let voters vote and find out the results
Administrator	See the list of vote events	Know what has been created
Voter	See the list of vote events that I am a voter of	
Administrator	Edit a vote event	Change certain settings of the vote event
Administrator	Delete vote events	Remove unneeded vote events
Administrator	View a list of internal voters for the vote event	Know who are the eligible internal voters
Administrator	View a list of external voters for the vote event	Know who are the eligible external voters
Administrator	Delete internal/external voters from their respective lists	Remove vote access from certain voters
Administrator	Add an internal voter to a vote event using their email	Give vote access to certain users
Administrator	Add voter IDs to the external voter list	Increase the number of external voters that can vote
Administrator	Choose if I want to have internal voters only, external voters only, or both for a vote event	I have a choice of having fewer things to manage
Administrator	View the list of candidates (projects) for the vote event	Know what voters are voting for
Administrator	Add candidates by project ID to the vote event	Add projects for voters to vote for
Administrator	Batch add projects based on common categories like cohort year and achievement level	Save time adding candidates one by one
Administrator	Delete candidates from the vote event	Remove ineligible candidates

Administrator	Create a vote event where candidates are displayed in a list	
Administrator	Set the minimum and maximum amount of votes a voter can have	Control how much each voter can vote
Administrator	Randomize the order of candidates	Possibly increase the fairness of the vote event
Administrator	Set my custom instructions for the vote event	Deliver instructions to the voters in the voting page
Administrator/Voter	See the phase of the vote event	Take action if needed
Voter	See a list of candidates	Vote by selecting from a list of candidates
Voter	See how many votes I have	Know how many votes to cast
Administrator	Publish/unpublish results anytime	Decide when voters get to see the results of the vote event
Administrator	Choose if I want to display rank, votes, points or point percentage in the results page	Control what stats the voters get to see
Administrator	Limit the results to only the top few candidates	Hide candidates that did not score well and only let voters focus on the top few
Administrator	Set role weights	Give more priority to votes based on importance
Administrator	Delete votes	Remove invalid votes if needed
Administrator	View the results of the vote event	
Administrator	View the full set of results of the vote event	
Voter	View the results of the vote event if published	

Administrator	Copy internal voters from another voter event	Reuse an existing list of internal voters
Administrator	Copy external voters from another voter event	Reuse an existing list of external voters
Administrator	Allow voters to manually register themselves	Let those who want to vote register themselves without having to add them myself
Administrator	Import internal voters with a csv file	Batch import internal voters after consolidating the list externally
Administrator	Import external voters with a csv file	Batch import external voters after consolidating the list externally
Administrator	Automatically generate external voter IDs	Instantly generate how many IDs I want
Internal voter	Register for vote events I am interested in	Vote in them
Administrator	Create a vote event where candidates are displayed in a gallery with images	
Administrator	Create a vote event where candidates are not displayed	
Voter	Be able to input the candidates I want to vote for	
Voter	Search for candidates	Vote more efficiently
Administrator	Search for voters, candidates and votes in their respective areas	Manage them more efficiently

C - User Study Tasks

1. Setting up a vote event (Take the role of an administrator creating your own vote event)
    A. Add a vote event with title: Vote event
    B. Set the duration of the vote event to be from the start of next month to the end of next month (select any number for hours and minutes)
    C. Add the vote event before configuring the items below
    D. Ensure that the vote event can handle both internal and external voters
        a. Do not copy voters from another vote event
    E. Add the user with email student@skylab.com as an internal voter
    F. Import the provided internal voter csv file internal-voters-template.csv
    G. Set a period for internal voter registration (any period you want)
    H. Add the external voter id acb123
    I. Import the provided external voter csv file external-voters-template.csv
    J. Generate 10 more external voter ids of length 6
    K. Add the project with an id of 1 as a candidate of the vote event
    L. Batch add all 2024 Apollo achievement level projects as candidates
    M. Set the following vote config: Gallery candidate display, minimum vote of 1 and maximum vote of 3, randomize the candidate order, add any instructions you want.

2. Cast votes in the vote event with with no candidate display (Take the role of a voter)
    A. Register for the vote event named: No candidate display vote event
        a. You are not in the list of internal voters, but registration is currently open. Register to be an internal voter.
    B. Submit votes in the vote event
        a. Vote for project ids: 2, 41, 67

3. Cast votes in the vote event with table view candidate display (Take the role of a voter)
    A. Submit votes in the vote event named: Table display vote event
        a. Vote for any project

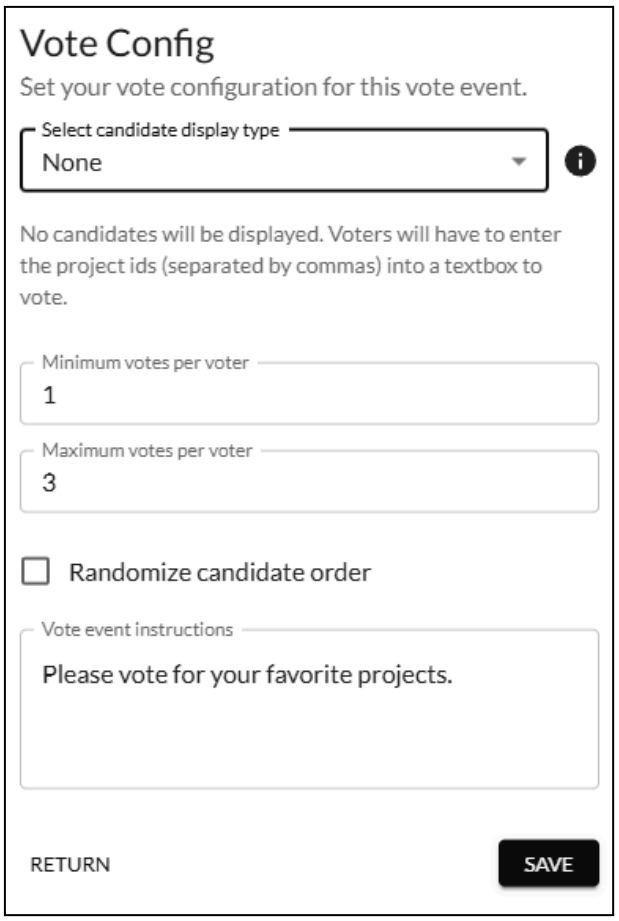
4. Cast votes in the vote event with gallery view candidate display (Take the role of a voter)
    A. Submit votes in the vote event named: Gallery display vote event
        a. Vote for any project
        b. The images shown are meant to be the project posters (currently fake data images)

5. Publish results of Table display vote event (Take the role of an administrator publishing the results for voters)
    A. Check the votes in the vote config tab
    B. Check the results in the results tab
    C. Adjust the role weights to what you think each role deserves
    D. Publish the results, hiding only the number of votes

6. View results (Take the role of a voter)
    A. View the results of the vote event named Table display vote event

D - System Screenshots of Primary Customization Features

Vote config modal



No candidate display voting page (None option)

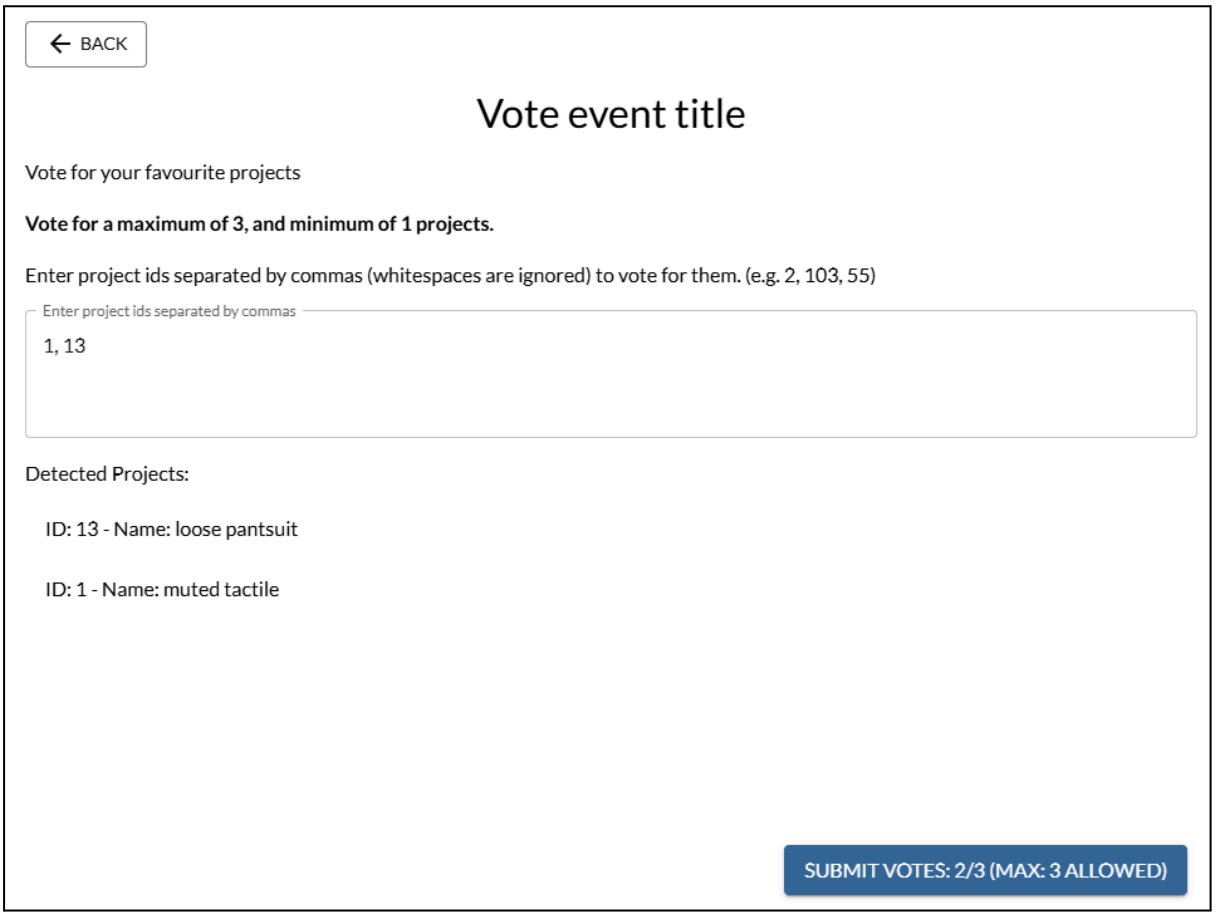
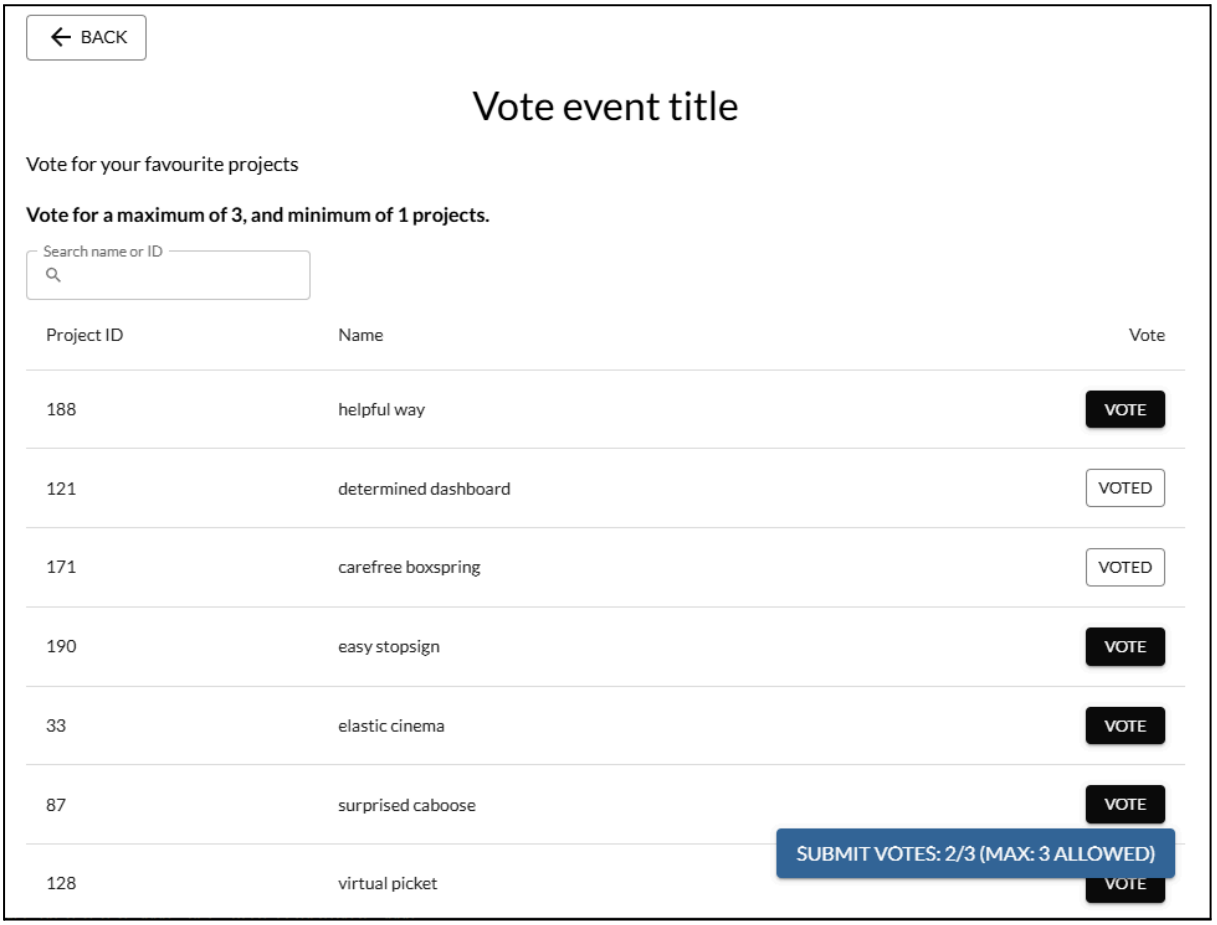
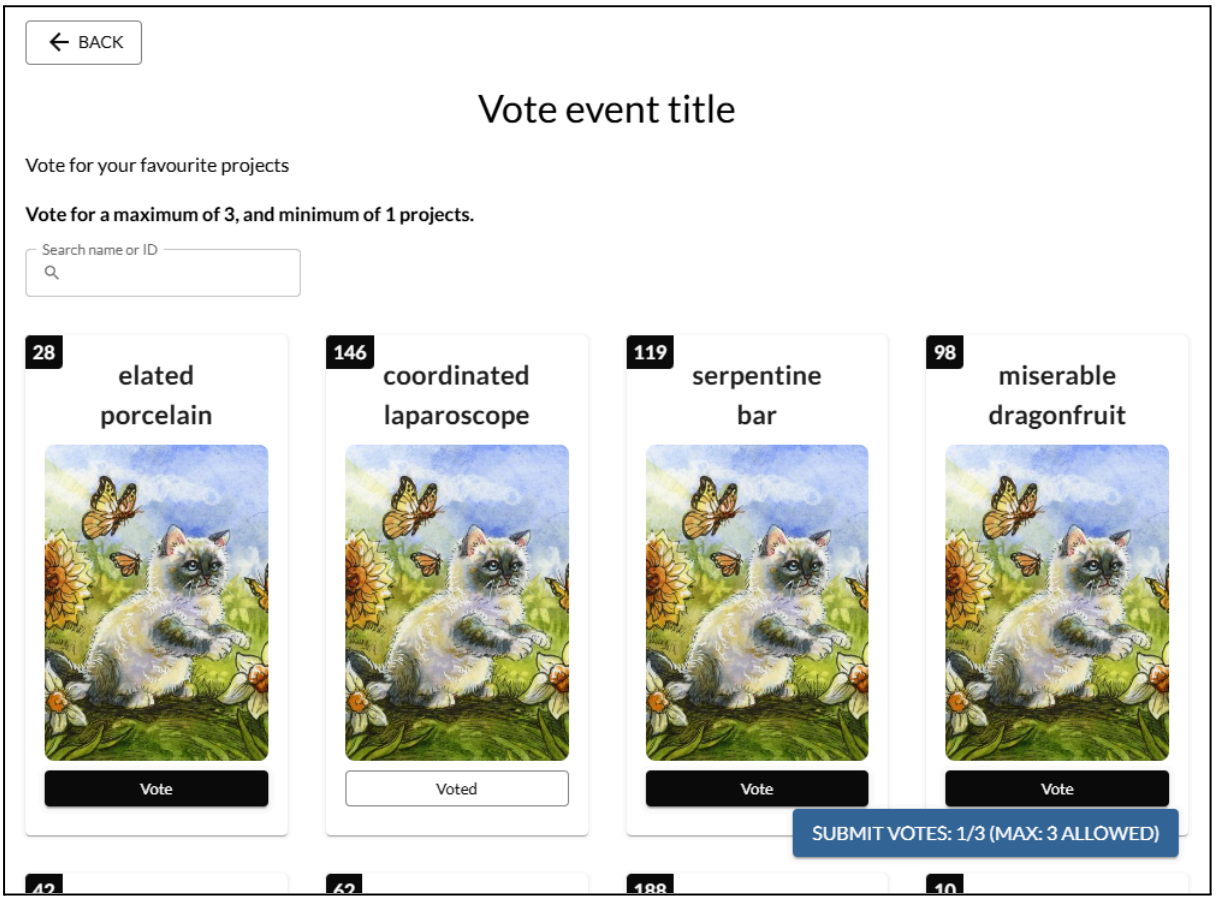


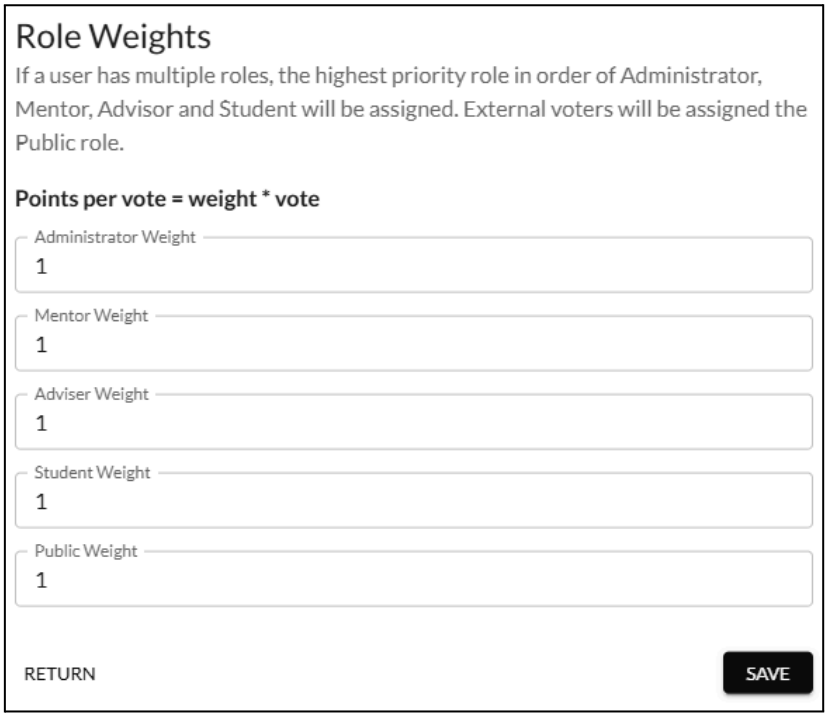
Table view candidate display voting page (Table option)



Gallery view candidate display voting page (Gallery option)



Role weights modal



Publish results modal